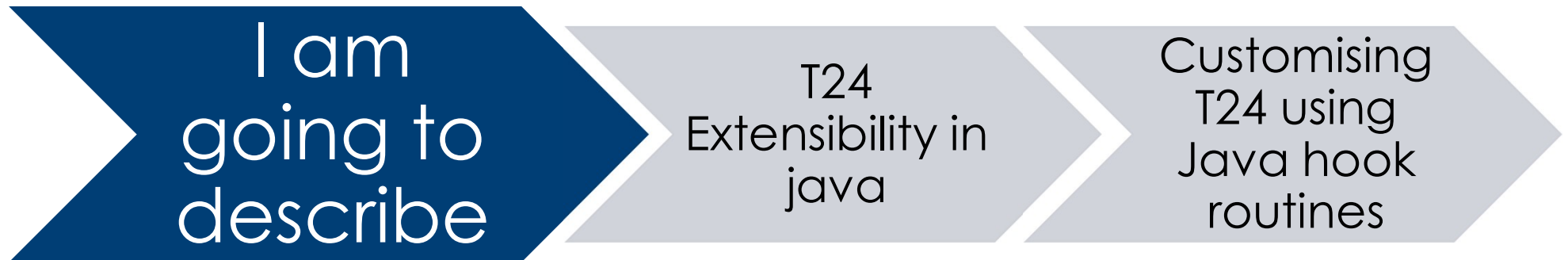


| T24 Extensibility in Java



TEMENOS
Learning Community

| Lesson Overview



| Your Course » Objective and Learning Outcomes

This course will introduce you to:

- T24 Extensibility in Java

In particular you will be able to:

- Understand T24 extensibility and customisation using Java
- Setup and Use Design Studio to Create and execute VERSION, ENQUIRY, SERVICE hooks
- Create and execute hooks attached to a local application
- Debug the Java code in Design Studio

| Your Course » Timetable



Day 1

- Introduction to T24 Extensibility in Java
- Setup Design Studio
- VERSION hooks



Day 2

- DEBUG the Java code in Design Studio
- ENQUIRY hooks
- SERVICE hooks
- Local application using EB.TABLE.DEFINITION



| Lesson 1. Introduction

T24 Extensibility in Java



TEMENOS
Learning Community

| Agenda - T24 Extensibility in Java

- Why do we need T24 Extensibility in Java ?
- APIs and Hooks
- T-Types
- DataAccess
- Complex classes
- Setup Design Studio

| Why do we need T24 Extensibility in Java ?

- Scalability and ownership
 - There are more java developers than jBC developers
 - Banks can use their own developers
- Simplify the T24 APIs
 - The developer should not need to know about STORE.END.ERROR, R.NEW or the different ways of storing an amount field in T24
- Governance
 - We can use the framework to protect t24. for e.g. prevent a select on a large table that can lead to performance issue for all users

| Prerequisites

- JD product must be installed

SPF	SYSTEM	
Products.129	JD	Java Development

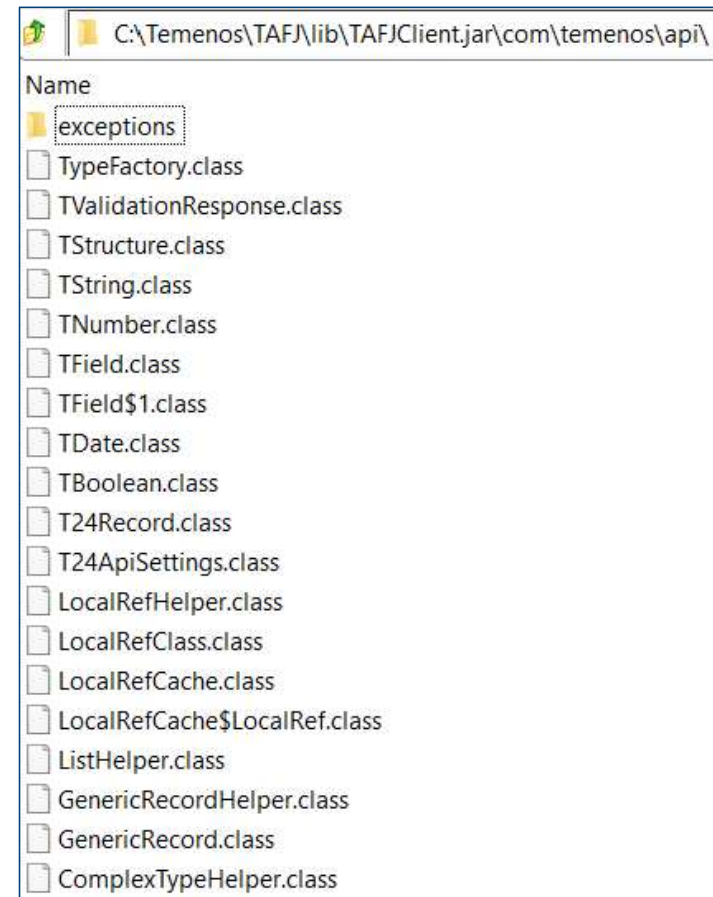
Restructure Tables.1	
Last Cache Reset Period.1	
Enquiry Sample Size	
Extensible Customisation	▼ No Yes
Ext Security Frame	
Deployment Mode	
Infinity	☒ [None] ☐ No ☐ Yes
Reserved 4	

| APIs and Hooks

- APIs are code Temenos has written that L3 developers can call
- Hooks are code L3 developers have written that T24 can call
- A developer can attach own logic to T24 to be called from core application exits using HOOKS
- Each hook has been provided with methods to carry out the task
- For e..g the Java developer will use the validateField method defined in `com.temenos.t24.api.hook.system.RecordLifeCycle` class for field validation exits

| 'T' types in T24

- Classes introduced by Temenos for Java developers in TAFJClient.jar
- TStructure
- TField
- TValidationResponse
- Many more like TString, TNumber, TBoolean, TDate ...



| 'T' types in T24 - TStructure

- The “**TStructure**” is a generic type containing a record object
- TStructure must be ‘cast’ to the correct record type
- Maps a jBC dynamic array to java object
- To access a record, the java developer must construct an instance of a record or complex type from the received parameter

```
public TValidationResponse validateRecord(String application, String recordId, TStructure record,
    TStructure lastLiveRecord) {
    FundsTransferRecord fr = new FundsTransferRecord(record);
    TField f1 = fr.getDebitCurrency();
    TField f2 = fr.getCreditCurrency();

    if (!f1.getValue().equals(f2.getValue()))
    {
        f2.setError("CREDIT CURRENCY IS NOT EQUAL TO DEBIT CURRENCY");
    }
    return fr.getValidationResponse();
}
```

| 'T' types in T24 - TField

TField offers getters and setters for Enrichment, Error, field values.

Every field in a record is an internal type **TField** (not String by default)

```
FTVersionDemo.java 88
package com.newbank;

import java.util.List;

public class FTVersionDemo extends RecordLifecycle {

    @Override
    public TValidationResponse validateRecord(String application, String recordId, TStructure record,
        TStructure lastLiveRecord) {
        FundsTransferRecord fr = new FundsTransferRecord(record);
        TField f1 = fr.getDebitCurrency();
        TField f2 = fr.getCreditCurrency();

        if (!f1.getValue().equals(f2.getValue()))
        {
            f2.setError("CREDIT CURRENCY IS NOT EQUAL TO DEBIT CURRENCY");
        }
        return fr.getValidationResponse();
    }
}
```

| 'T' types in T24 - TValidationResponse

Record validation is a common mechanism in T24 to allow custom validation on records.

```
FTVersionDemo.java  ⌕
package com.newbank;

import java.util.List;

public class FTVersionDemo extends RecordLifecycle {

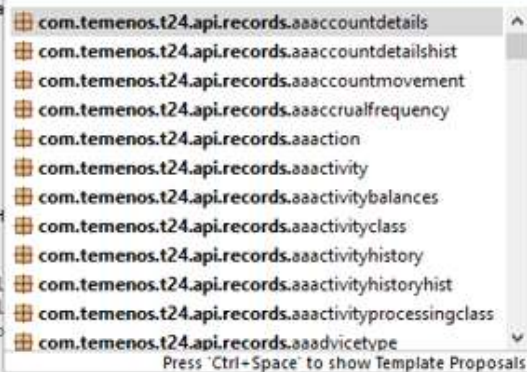
    @Override
    public TValidationResponse validateRecord(String application, String recordId, TStructure record,
        TStructure lastLiveRecord) {
        FundsTransferRecord fr = new FundsTransferRecord(record);
        TField f1 = fr.getDebitCurrency();
        TField f2 = fr.getCreditCurrency();

        if (!f1.getValue().equals(f2.getValue()))
        {
            f2.setError("CREDIT CURRENCY IS NOT EQUAL TO DEBIT CURRENCY");
        }
        return fr.getValidationResponse();
    }
}
```

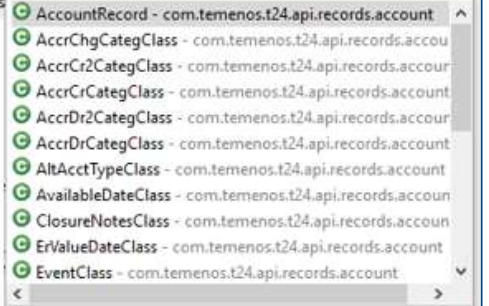
DataAccess

- The system.DataAccess class is a utility class to read, select and access data in T24.
- The api.records package is used to hold records from T24 applications

```
import com.temenos.t24.api.hook.system.Enquiry;  
import com.temenos.t24.api.records.  
import com.temenos.t24.api.system.DataAccess
```



```
import com.temenos.t24.api.hook.system.Enquiry;  
import com.temenos.t24.api.records.account.  
import com.temenos.t24.api.system.DataAccess
```



```
DataAccess da = new DataAccess(this);  
AccountRecord AccRec = new AccountRecord(da.getHistoryRecord("ACCOUNT", currentId));  
TField title = AccRec.getAccountTitle1(0);  
return title.toString();
```

| 'T' types in T24 – T24Context

- T24Context is the way to establish a connection between Java application and T24
- TAFJClient.jar is required in JAVA_PROJECT to establish the connection.(available in TAFJ_HOME/lib).
- T24Context is used to set the credentials that pass through T24 Validation.
- Hook routines do not need a T24Context to connect to T24.

| 'T' types in T24 – T24Context

```
package com.newbank;

import com.temenos.t24.api.complex.st.customerapi.PersonalInfo;
import com.temenos.t24.api.party.Customer;
import com.temenos.tafj.api.client.impl.T24Context;

public class GetT24Context {

    public static void main(String[] args) {
        System.setProperty("tafj.home",
            "C:/Temenos/TAFJ");
        T24Context ctx = new T24Context("tafj");
        ctx.setPassword("123456");
        ctx.setUser("INPUTT");

        Customer customer = null;
        customer = new Customer(ctx);
        customer.setCustomerId("100352");

        PersonalInfo personalInfo = customer.getPersonalInfo();
        System.out.println("Nationality " + personalInfo.getNationality());
        System.out.println("Residence " + personalInfo.getResidence());
        System.out.println("Date of Birth " + personalInfo.getDateOfBirth());
    }
}
```

TAFJ
properties file

OUTPUT

```
(OFS.INITIALISE.SOURCE) : JAVA.FRAMEWORK
Nationality AU
Residence AU
Date of Birth 19711225
```

Complex class in T24

- No need to set/get related information in separate parameters.
- For e.g. Customer PersonalInfo is made up of DOB, nationality and residence

```
package com.newbank;

import com.temenos.t24.api.complex.st.customerapi.PersonalInfo;
import com.temenos.t24.api.party.Customer;
import com.temenos.tafj.api.client.impl.T24Context;

public class GetT24Context {

    public static void main(String[] args) {
        System.setProperty("tafj.home",
            "C:/Temenos/TAFJ");
        T24Context ctx = new T24Context("tafj");
        ctx.setPassword("123456");
        ctx.setUser("INPUTT");

        Customer customer = null;
        customer = new Customer(ctx);
        customer.setCustomerId("100352");

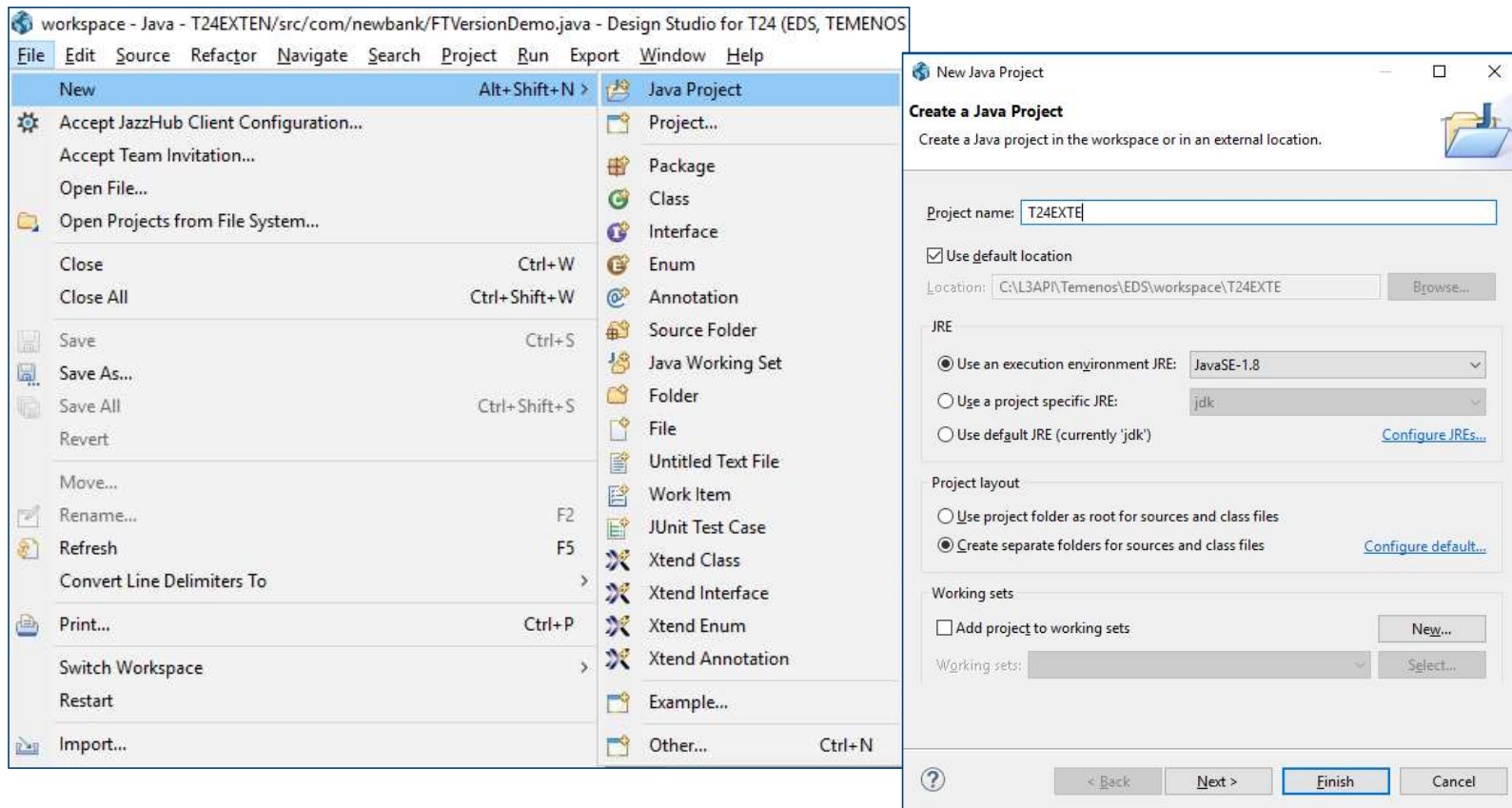
        PersonalInfo personalInfo = customer.getPersonalInfo();
        System.out.println("Nationality " + personalInfo.getNationality());
        System.out.println("Residence " + personalInfo.getResidence());
        System.out.println("Date of Birth " + personalInfo.getDateOfBirth());
    }
}
```

```
PersonalInfo
{
    dateOfBirth : date
    nationality : string
    residence : string
}
```

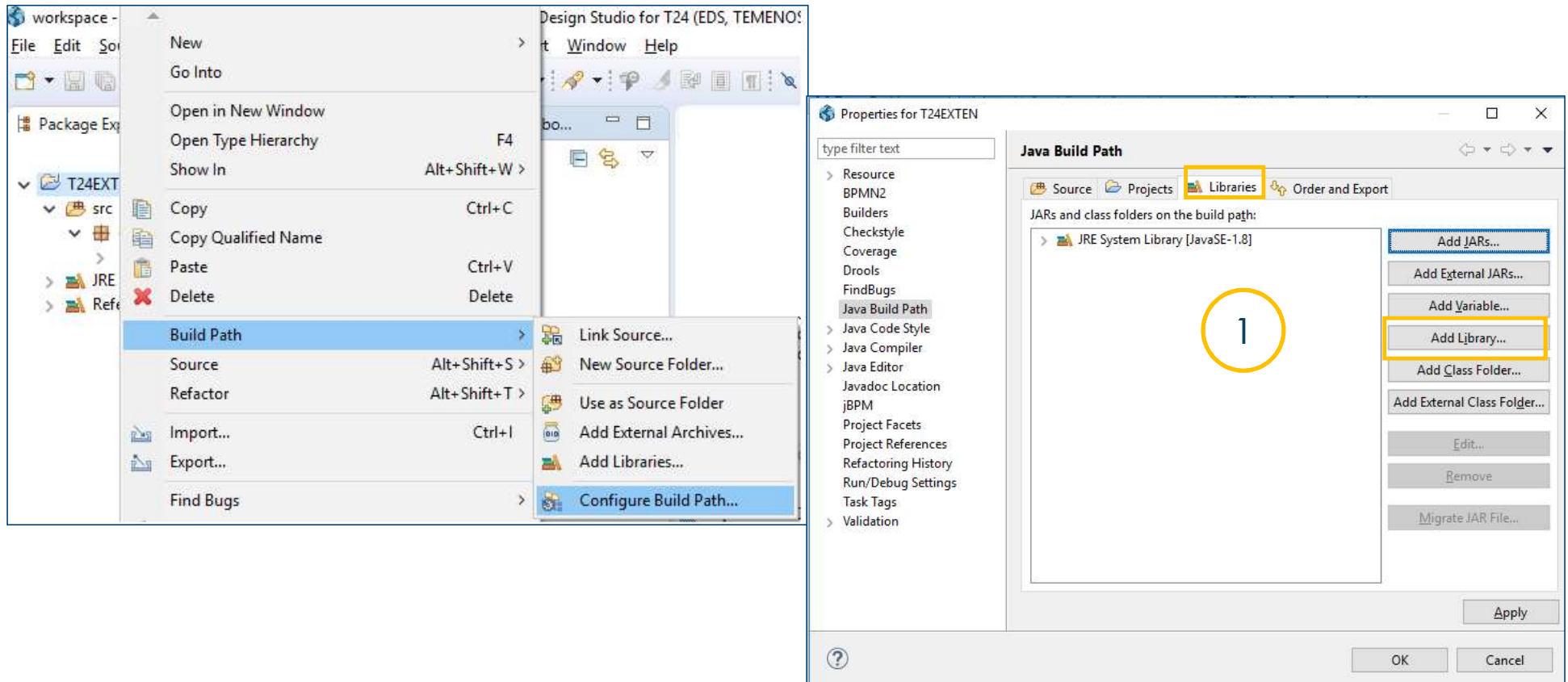
```
(OFS.INITIALISE.SOURCE) : JAVA.FRAMEWORK
Nationality AU
Residence AU
Date of Birth 19711225
```

Setup - Creating java project in Design Studio

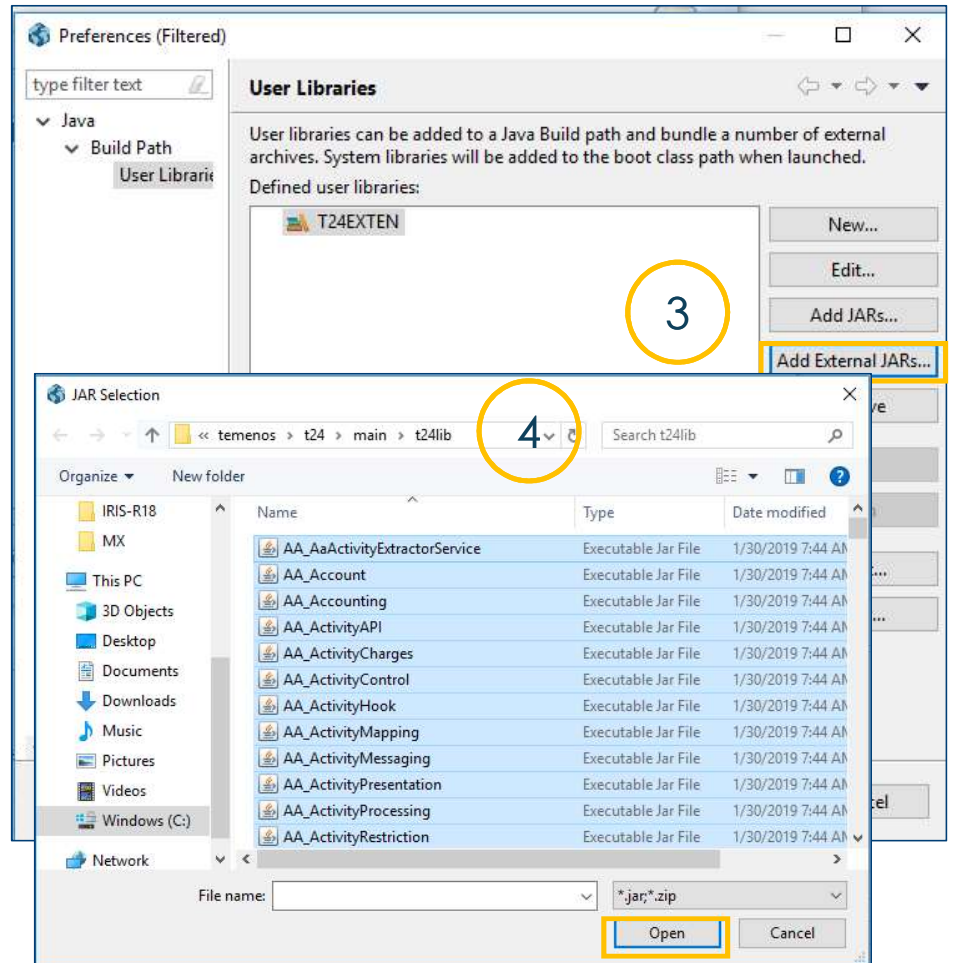
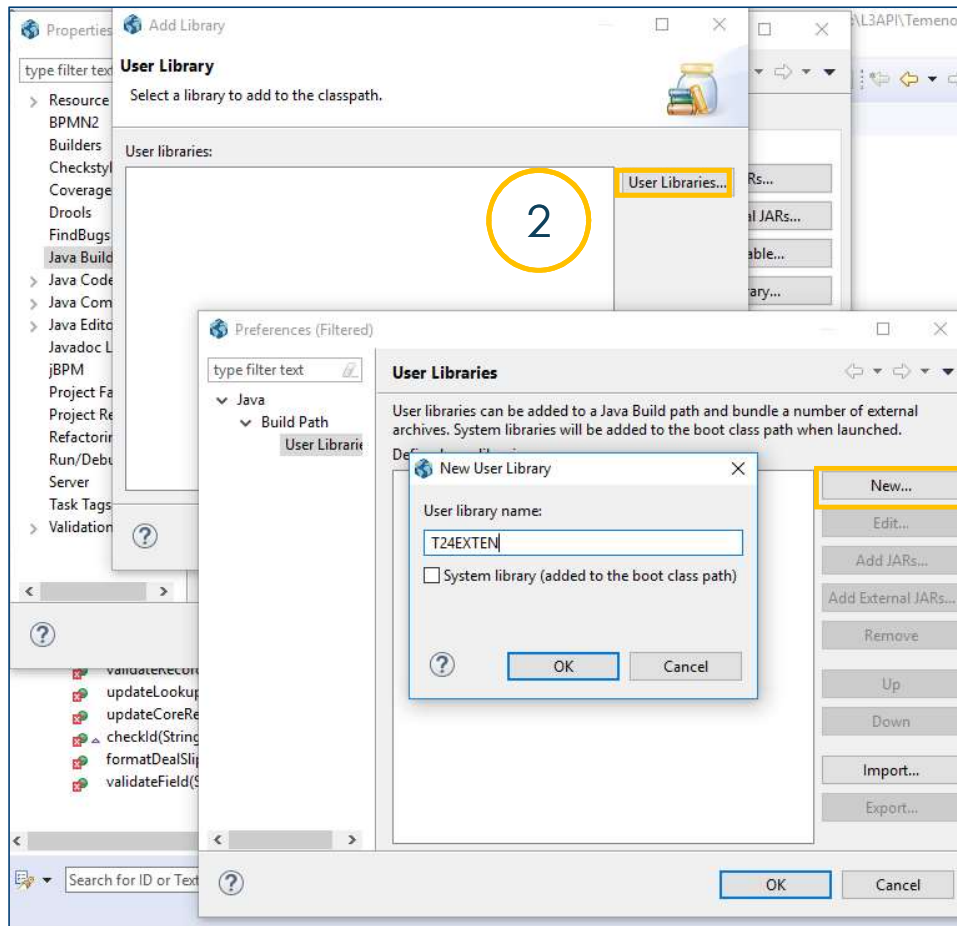
- File → New → Java Project (ProjectName)



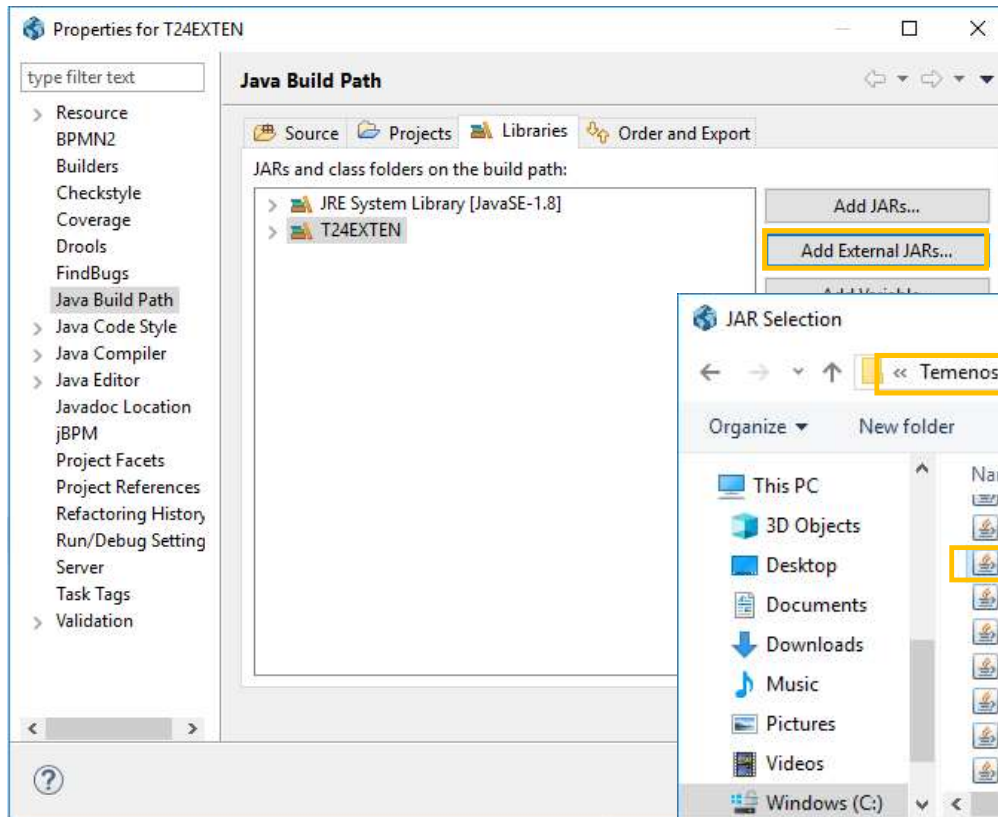
Setup - Configure Build Path – T24 Precompile



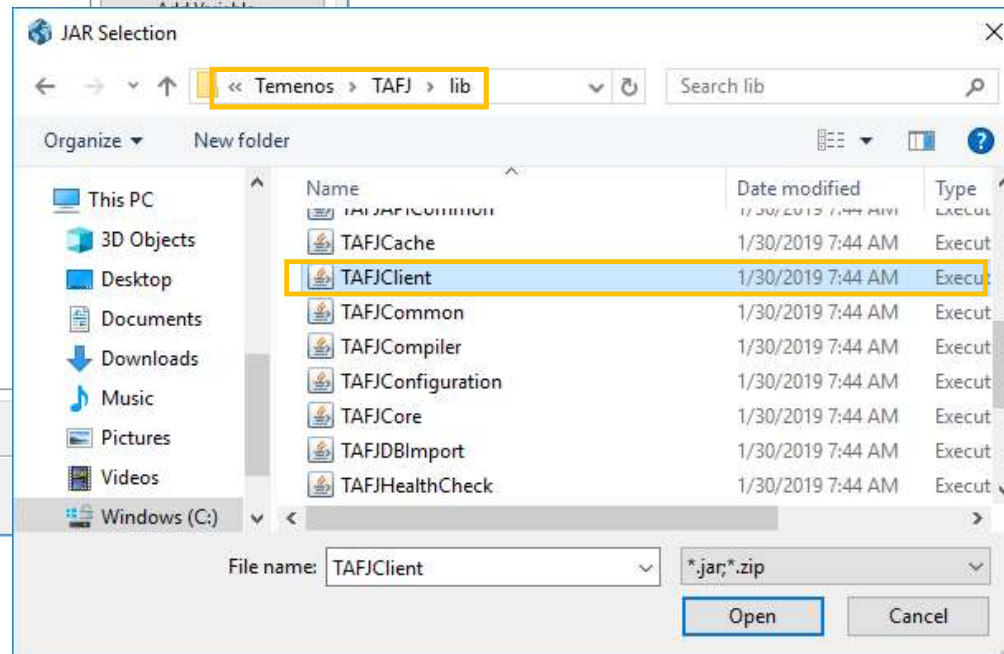
Configure Build Path – T24 Precompile



Configure Build Path - TAFJClient

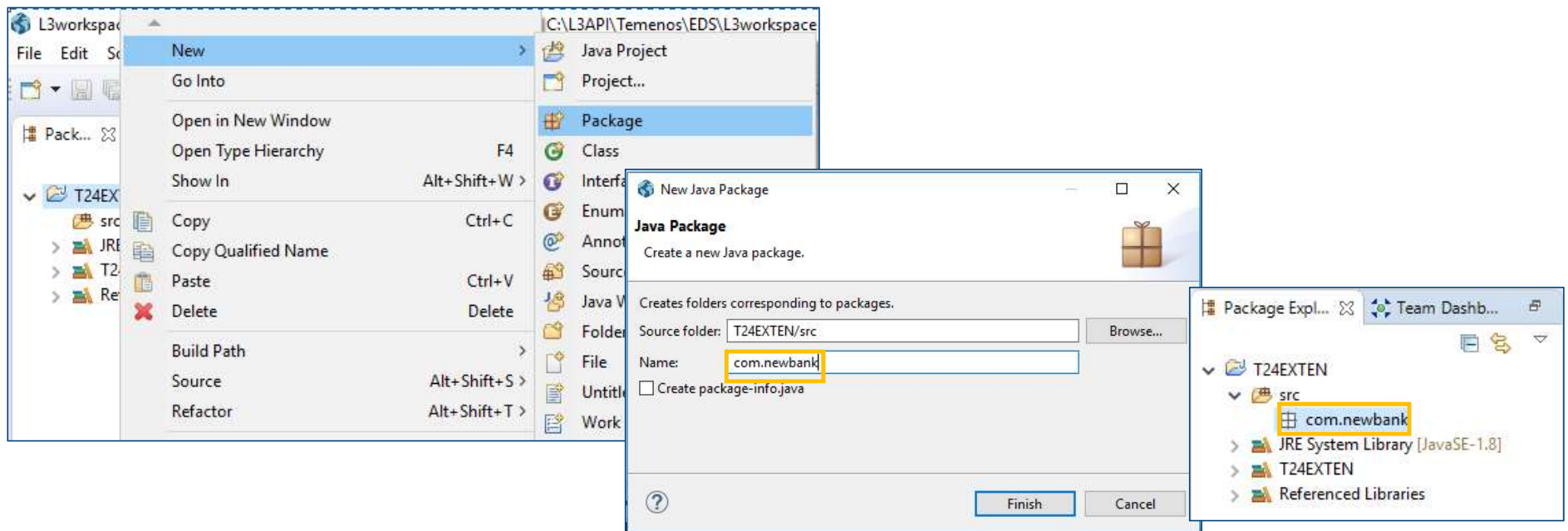


- Add TAFJClient.jar available in TAFJ_HOME/lib to the Java Build Path
- This is needed to establish connection and access the Datatypes



Create package

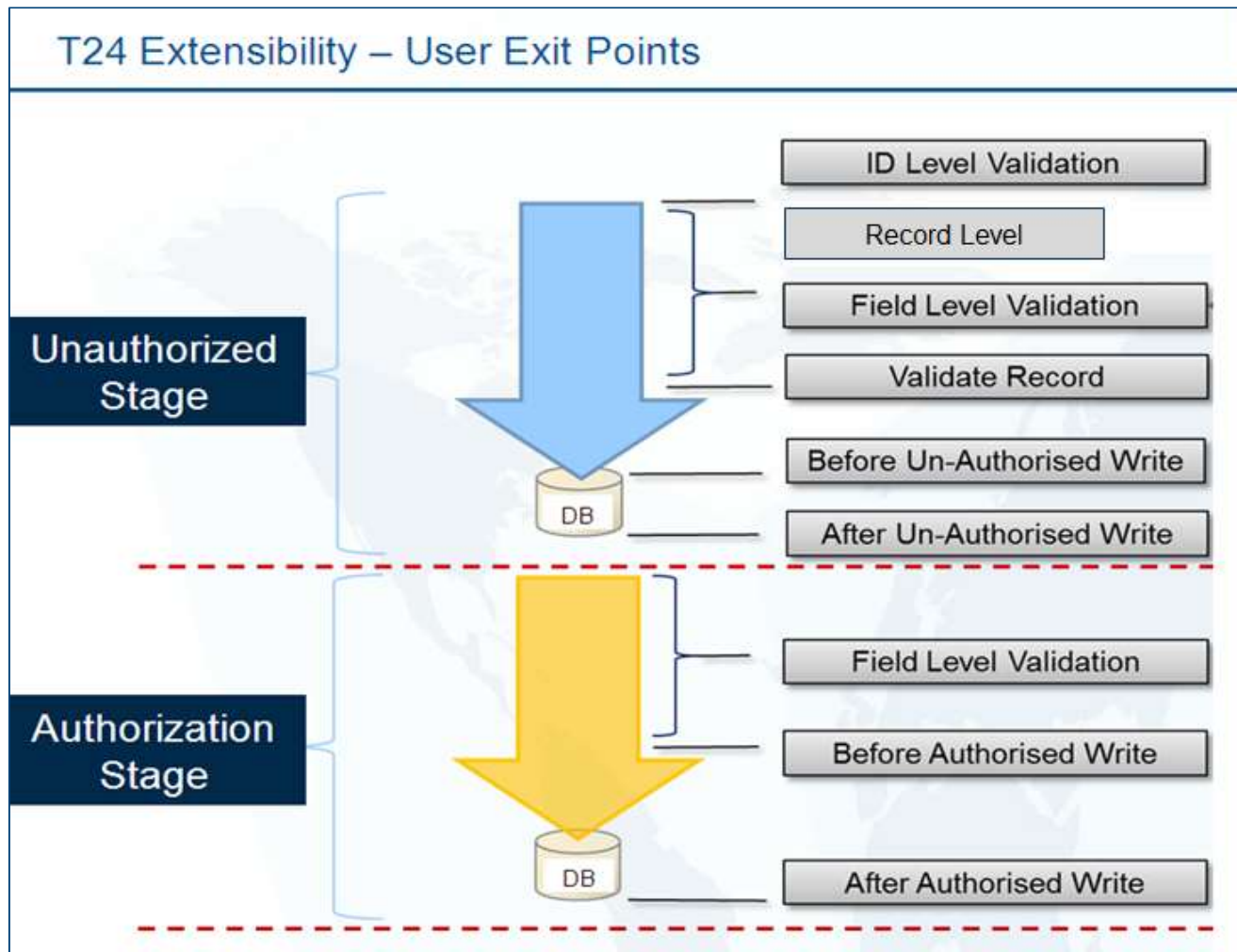
- Java package is a mechanism for organizing Java classes into namespaces similar to the T24 modules.
- All classes go into a package
- Right Click on your project → New → Package → PackageName.



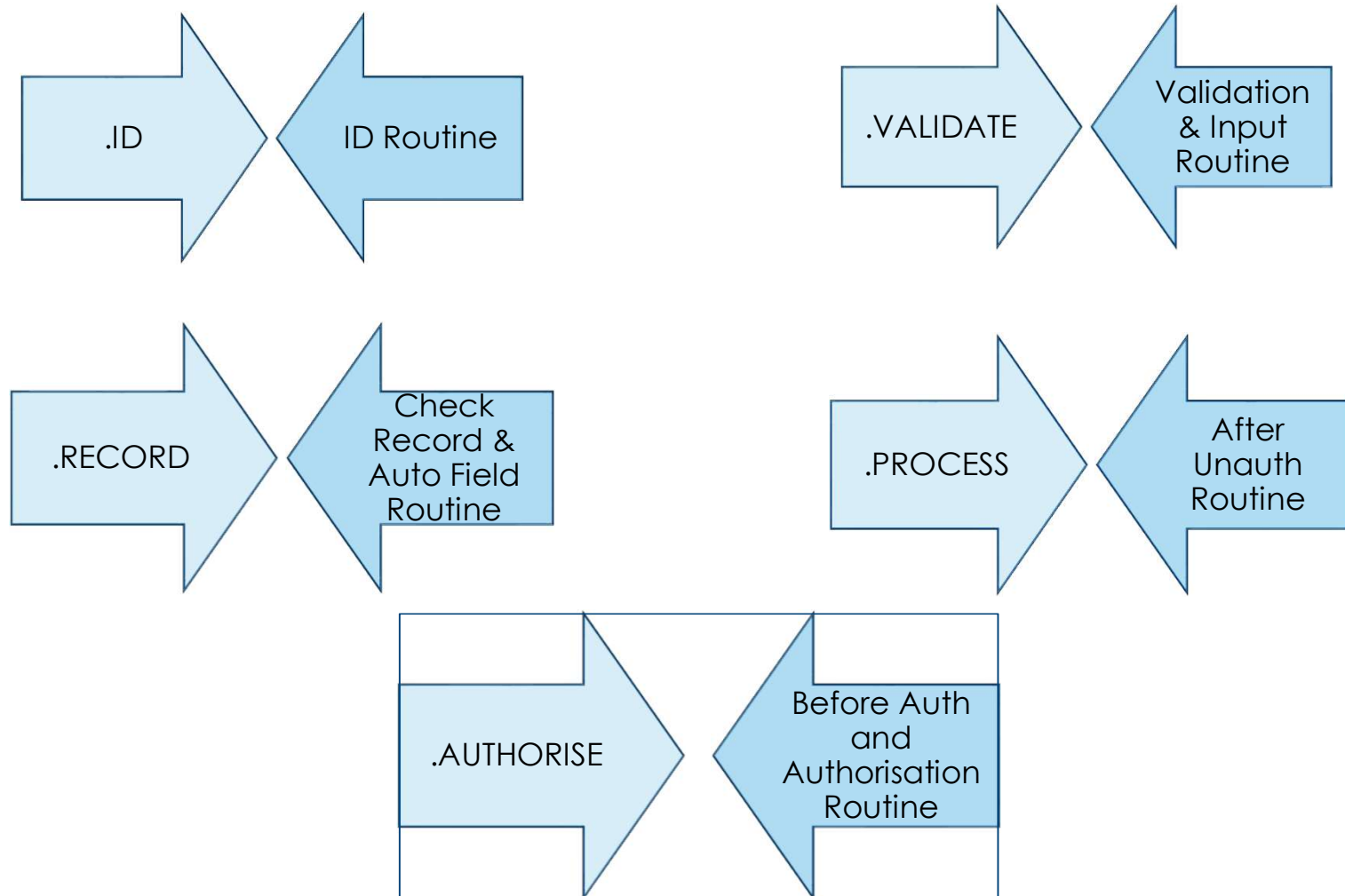
Lesson 2. VERSION Hooks in Java

T24 Extensibility in Java

T24 Extensibility



Routine Invocation Stages in the Template Life Cycle



| VERSION HOOKS in T24

- Version hooks are attached to a VERSION
- **com.temenos.t24.api.hook.system.RecordLifecycle** has the following methods
 - checkId
 - defaultFieldValues
 - formatDealSlip
 - getDataDefinition
 - updateCoreRecord
 - updateLookupTable
 - validateField
 - validateRecord

| DEMO – VERSION HOOK

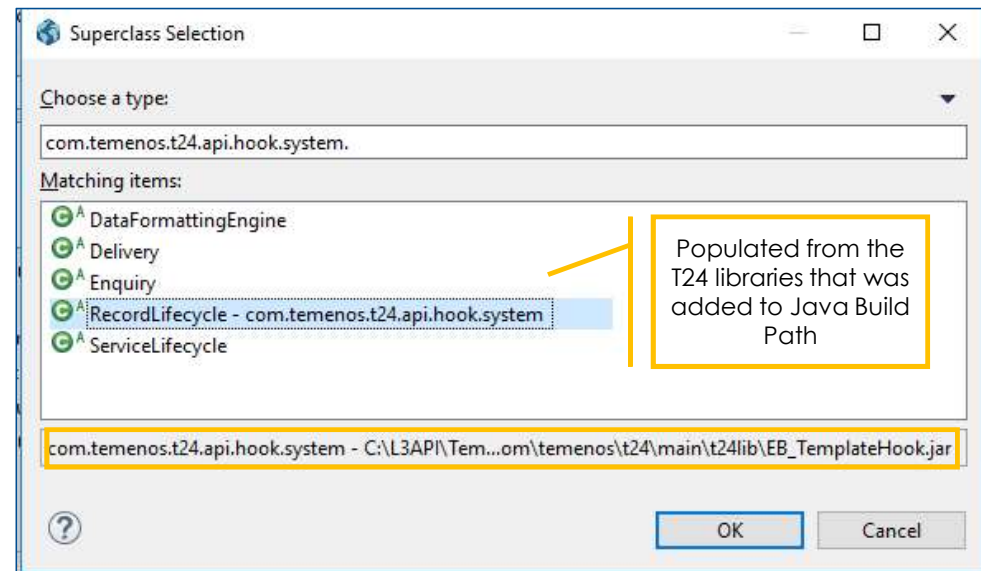
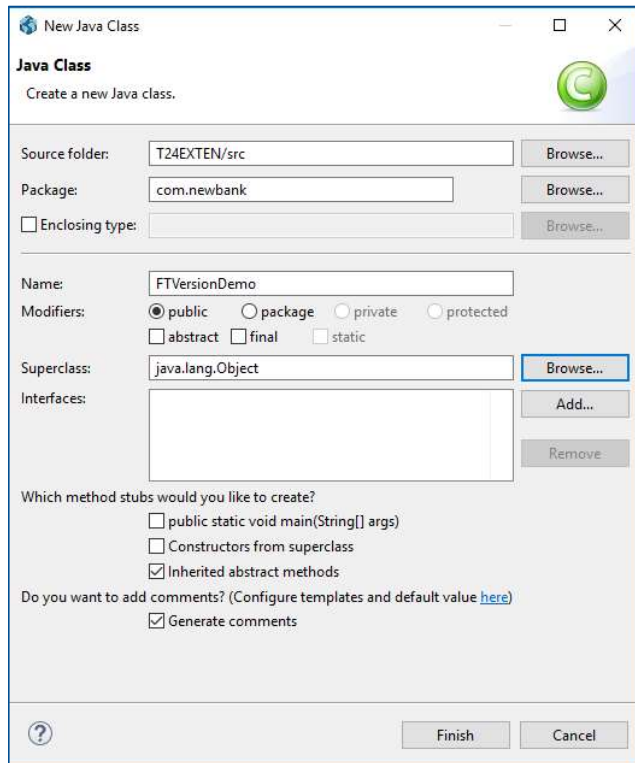
For a FUNDS.TRANSFER transaction, error must be raised when the DEBIT.CURRENCY and CREDIT.CURRENCY are not equal

Solution

1. Create a class that extends **RecordLifecycle** and override the `validateRecord()`
2. Create JAR, add in module.xml and restart jBoss
3. Make an entry in EB.API
4. Create a VERSION and attach the EB.API record to the Input routine to validate if DEBIT.CURRENCY and CREDIT.CURRENCY are the same

Step 1 – Create class

- Right Click on your Package → New → Class

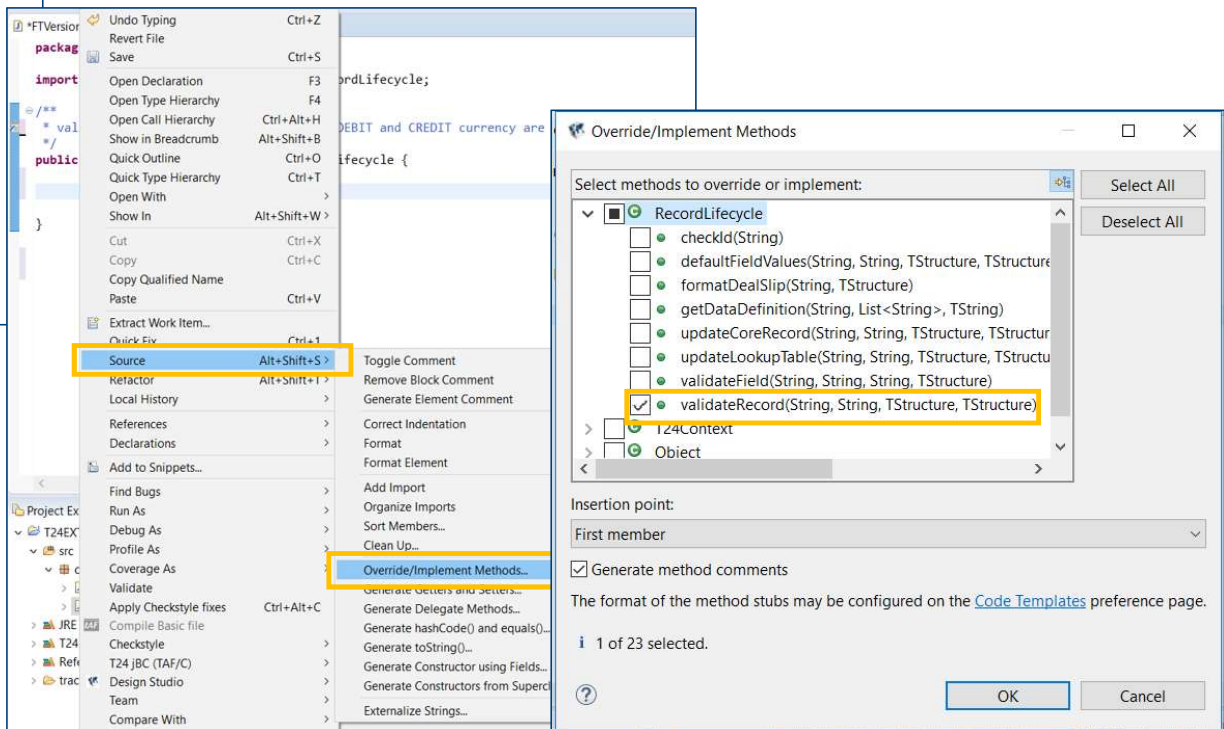


Step 1 – implement validateRecord

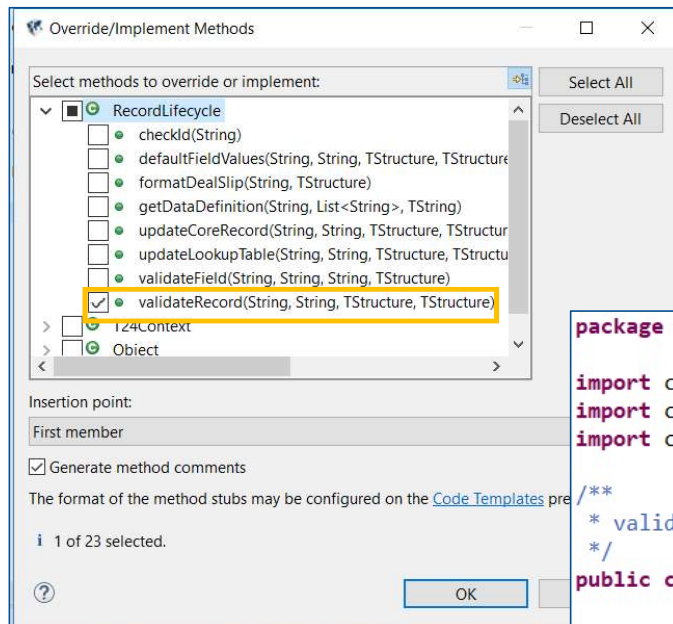
```
package com.newbank;

import com.temenos.t24.api.hook.system.RecordLifecycle;

/**
 * TODO: Document me!
 *
 * @author sabithakr
 */
public class FTVersionDemo extends RecordLifecycle {
}
```



Step 1 – implement validateRecord



```
package com.newbank;

import com.temenos.api.TStructure;
import com.temenos.api.TValidationResponse;
import com.temenos.t24.api.hook.system.RecordLifecycle;

/**
 * validateRecord is used to check if the DEBIT and CREDIT currency are the same
 */
public class FTVersionDemo extends RecordLifecycle {

    @Override
    public TValidationResponse validateRecord(String application, String recordId, TStructure record,
        TStructure lastLiveRecord) {
        // TODO Auto-generated method stub
        return super.validateRecord(application, recordId, record, lastLiveRecord);
    }
}
```

Step 1 – implement validateRecord

```
package com.newbank;

import com.temenos.api.TField;
import com.temenos.api.TStructure;
import com.temenos.api.TValidationResponse;
import com.temenos.t24.api.hook.system.RecordLifecycle;
import com.temenos.t24.api.records.fundstransfer.FundsTransferRecord;

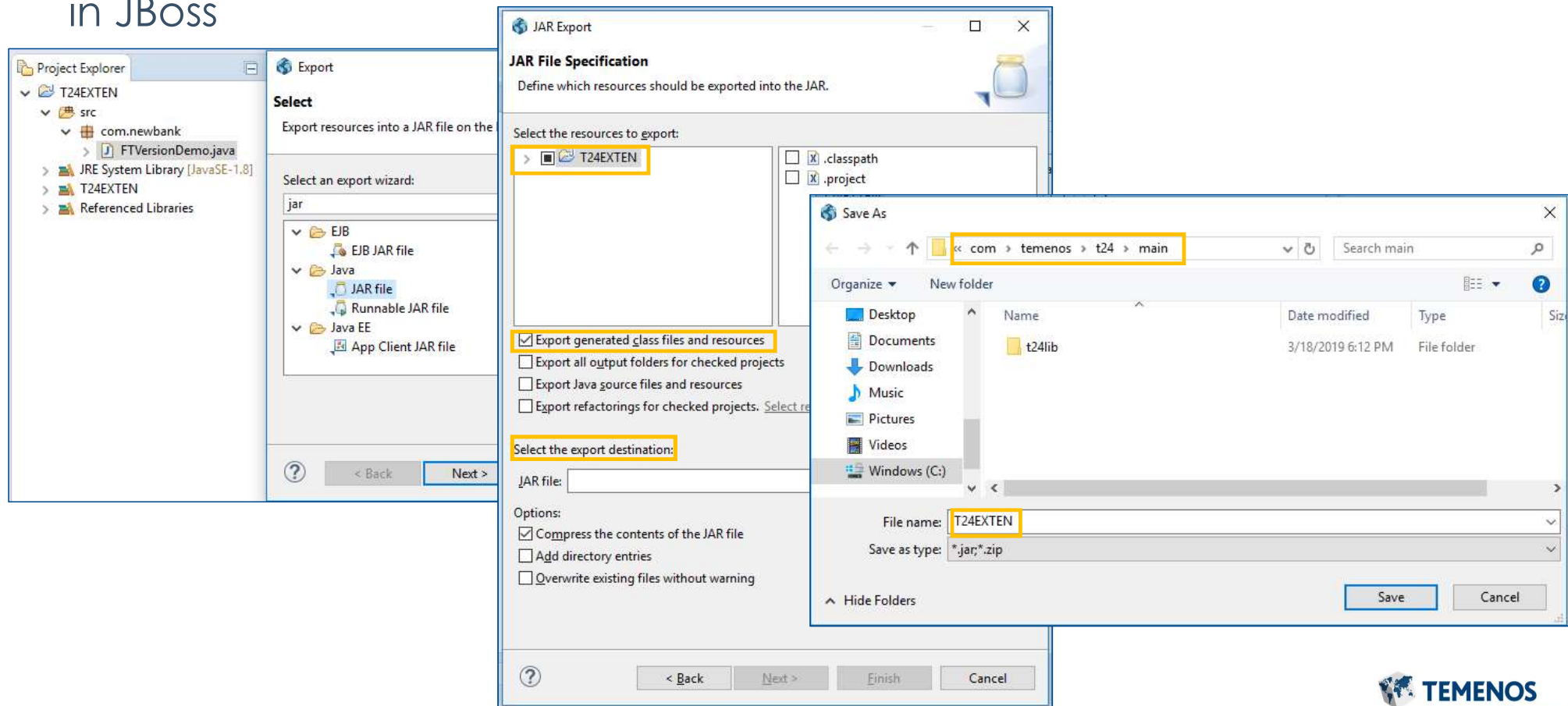
/**
 * validateRecord is used to check if the DEBIT and CREDIT currency are the same
 */
public class FTVersionDemo extends RecordLifecycle {

    @Override
    public TValidationResponse validateRecord(String application, String recordId, TStructure record,
        TStructure lastLiveRecord) {
        FundsTransferRecord fr = new FundsTransferRecord(record);
        TField f1 = fr.getDebitCurrency();
        TField f2 = fr.getCreditCurrency();

        if (!f1.getValue().equals(f2.getValue()))
        {
            f2.setError("CREDIT CURRENCY IS NOT EQUAL TO DEBIT CURRENCY");
        }
        return fr.getValidationResponse();
    }
}
```

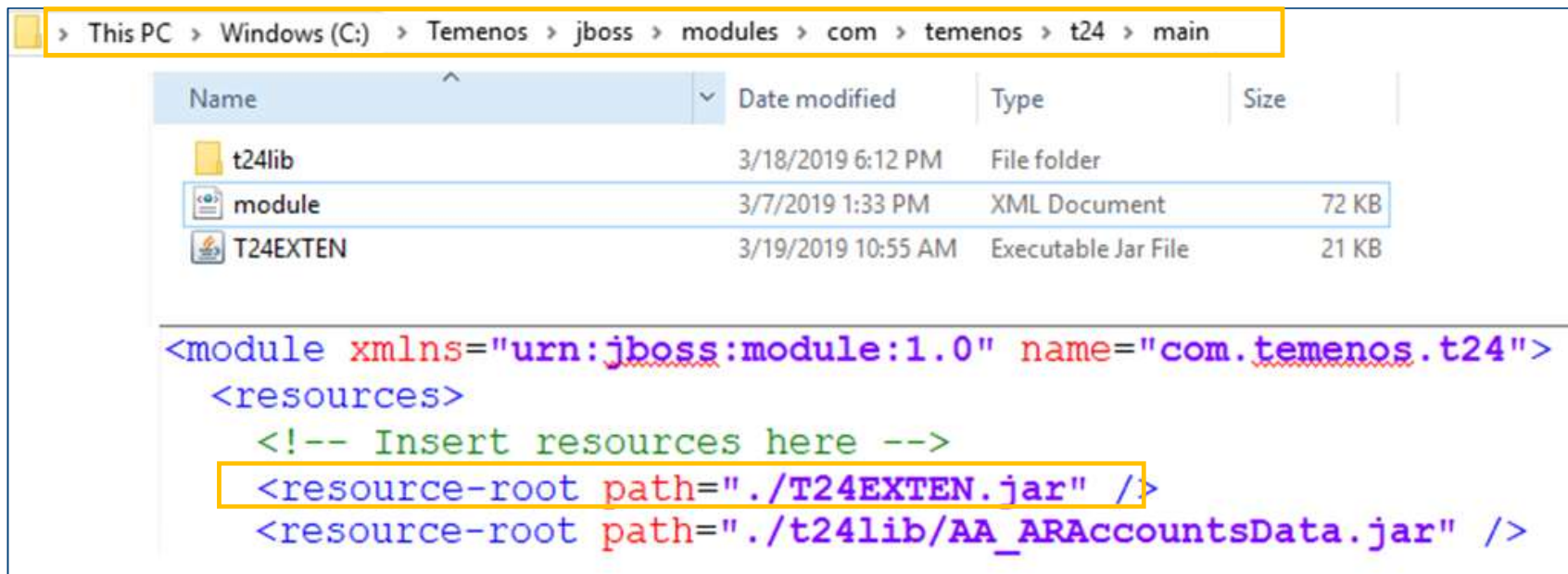
Step 2 – Create the JAR and add to module.xml

- The .class file must be exported as a JAR and added to the module.xml in JBoss



Step 2 – Create the JAR and add to module.xml

- Add the JAR to the jBoss module.xml and restart jBoss



File Explorer Path: This PC > Windows (C:) > Temenos > jboss > modules > com > temenos > t24 > main

Name	Date modified	Type	Size
t24lib	3/18/2019 6:12 PM	File folder	
module	3/7/2019 1:33 PM	XML Document	72 KB
T24EXTEN	3/19/2019 10:55 AM	Executable Jar File	21 KB

```
<module xmlns="urn:jboss:module:1.0" name="com.temenos.t24">
  <resources>
    <!-- Insert resources here -->
    <resource-root path="./T24EXTEN.jar" />
    <resource-root path="./t24lib/AA_ARAccountsData.jar" />
  </resources>
</module>
```

| Step 3 – EB.API

- Hooks are invoked when a transaction is validated/committed
- Input Hooks are attached to the field INPUT.ROUTINE in the VERSION application
- All hooks must have an entry in EB.API

EB.API FTVERSION.DEMO	
Description	EN FT Version Hook
Protection Level *	☒ Full ☐ Partial ☐ None
Source Type *	☐ Basic ☐ Java ☐ Hook ☒ Method
Java Method	validateRecord
Java Class	FTVersionDemo
Java Package	com.newbank

Step 4 – Create/Modify version

- Modify the version FUNDS.TRANSFER,AC

Version Ds	FUNDS.TRANSFER,AC	
D Slip Trigger	☐ OI ☐ Rq	
Input Routine.1	FTVERSION.DEMO	FT Version Hook ▼ +
Auth Routine.1		+

EB.API record is specified as the INPUT.ROUTINE

Launching the VERSION

- Error is raised when DEBIT.CURRENCY is not equal to CREDIT.CURRENCY

Transfer Between Accounts

FT18107K2MR8

✓

⏸

⬆

ℹ

- Please Select

GO

Transfer Between Accounts

Audit

Debit Account *

19461

⌵

⋮

WILLIAM HEWLETT

Debit Currency

USD

US Dollar

⌵

⋮

Debit Value Date

17 APR 2018

📅

17 APR 2018

Cheque Type

⌵

Ordered By.1

+

Credit Account *

11223

NIKE

⌵

⋮

Credit Currency

GBP

Pound Ster

⌵

⋮

CREDIT CURRENCY IS NOT EQUAL TO DEBIT CURRENCY

Debit Amount

123.00

Debit Narrative

Cheque Number

Credit Amount

Please resolve the errors below to proceed:

✗ Credit Currency: CREDIT CURRENCY IS NOT EQUAL TO DEBIT CURRENCY

Launching the VERSION

- Error is not raised when DEBIT.CURRENCY equal to CREDIT.CURRENCY

Transfer Between Accounts FT18107K2MR8

✓ 📄 ⏸ ⬆ ⓘ - Please Select

Information: Field has been validated ✕

Transfer Between Accounts **Audit**

Debit Account *	19461 WILLIAM HEWLETT		
Debit Currency	USD US Dollar	Debit Amount	123.00
Debit Value Date	17 APR 2018	Debit Narrative	
Cheque Type		Cheque Number	
Ordered By.1			
Credit Account *	21261 ZURICH EQUITY F		
Credit Currency	USD US Dollar	Credit Amount	
Credit Value Date	17 APR 2018	Credit Narrative	

Lesson 3. DEBUG Java code in Design Studio

T24 Extensibility in Java

DEMO – DEBUG the Java code

- The Design Studio Java IDE provides many debugging tools and views grouped in the Debug Perspective
- To debug the program, define breakpoints. By adding breakpoints in the source code we can specify where the execution of the program should pause
- To set breakpoints in the source code double click on the small left margin in the source code editor.
- DS uses eclipse standard for debugging
 - Step In : F5
 - Step Over : F6
 - Step Out : F7
 - Continue : F8



| **DEBUG the Java code**

- Remote Debug jBoss from Design Studio
- Set JAVA_OPTS

For Windows

- SET JAVA_OPTS=-Xdebug -Xnoagent -Xrunjdwp:transport=dt_socket, address=8787,server=y,suspend=n%JAVA_OPTS%

For Linux

- JAVA_OPTS="-Xdebug -Xnoagent -Xrunjdwp:transport=dt_socket, address=8787,server=y,suspend=n\$JAVA_OPTS"

DEBUG the Java code

- Create the DEBUG configuration

The screenshot illustrates the process of creating a new debug configuration in the Eclipse IDE. It is divided into three main sections:

- Top Left (Run Menu):** The 'Run' menu is open, and 'Debug Configurations...' is highlighted. Other options like 'Run', 'Debug', and 'Coverage' are also visible.
- Bottom Left (Toolbar):** The 'Debug' icon (a green bug) is highlighted in the toolbar. Below it, the 'Debug Configurations...' button is also highlighted.
- Right (Configuration Dialog):** The 'Create, manage, and run configurations' dialog is open. The 'New launch configuration' button is highlighted. The configuration is named 'JAVADEBUG'. The project is set to 'T24EXTEN'. The connection type is 'Standard (Socket Attach)'. The host is 'localhost' and the port is '8787'. The 'Allow termination of remote VM' checkbox is unchecked.

The list of configurations on the left side of the dialog includes: Generic Server(Extern..., HTTP Preview, J2EE Preview, Java Applet, Java Application, JBC Program, JBC Remote, JUnit, JUnit Plug-in Test, Maven Build, MWE Workflow, Mwe2 Launch, Node.js Application, OSGi Framework, Remote Java Applica, JAVADEBUG, and Remote JavaScript.

DEBUG the java code

Transfer Between Accounts FT18107370J1

Transfer Between Accounts Audit

Debit Account * 19461
WILLIAM HEWLETT

Debit Currency USD US Dollar Debit Amount 123

Debit Value Date DD MMM YYYY

Cheque Type

Ordered By.1

Credit Account * 11223

Debugger view

Variables and expressions view

The current instruction pointer

Debugger

Thread [default task-15] (Suspended (breakpoint at line 20 in FTVersionDemo))

- FTVersionDemo.validateRecord(String, String, TStructure, TStructure) line: 20
- NativeMethodAccessorImpl.invoke0(Method, Object, Object[]) line: not available [native method]
- NativeMethodAccessorImpl.invoke(Object, Object[]) line: 62
- DelegatingMethodAccessorImpl.invoke(Object, Object[]) line: 43
- Method.invoke(Object, Object...) line: 498
- component_EB_TemplateHook_19.cl.validateRecord(Object, Object, Object, Object, Object) line: not available
- EB_TEMPLATE_HOOK_VALIDATE_RECORD_INVOKER.cl.main() line: not available
- EB_TEMPLATE_HOOK_VALIDATE_RECORD_INVOKER.cl.invoke(Object, Object, Object) line: not available
- EB_TEMPLATE_HOOK_VALIDATE_RECORD_INVOKER.cl.invoke(Object...) line: not available
- component EB_TemplateHook_19.cl.validateRecordInvoker(Object, Object, Object) line: not available

FTVersionDemo.java GetT24Context.java FUNDS.TRANSFER.AC.version NativeMethodAccessorImpl.class

```
@Override
public TValidationResponse validateRecord(String application, String recordId, TStructure record,
    TStructure lastLiveRecord) {
    FundsTransferRecord fr = new FundsTransferRecord(record);
    TField f1 = fr.getDebitCurrency();
    TField f2 = fr.getCreditCurrency();

    if (!f1.getValue().equals(f2.getValue()))
    {
        f2.setError("CREDIT CURRENCY IS NOT EQUAL TO DEBIT CURRENCY");
    }
    return fr.getValidationResponse();
}
```

Variables

Name	Value
recordId	"FT18107370J1" (id=514)
record	TStructure (id=515)
lastLiveRecord	TStructure (id=520)
fr	FundsTransferRecord (id=528)
f1	TField (id=531)

USD

Outline

- com.newbank
- FTVersionDemo
- validateRecord(String, String, TStructure, TStructure): TValid

Practice – Version Hook CheckRec

- Create a VERSION for ACCOUNT application that allows user to edit records belonging to CATEGORY 1001 only
- Expected Output

Error must be raised for ACCOUNT 74047 as it belongs to CATEGORY 6001

TEMENOS ACCOUNT,CAT1001 | 74047

Runtime Exception Error This version can be used to edit records in category 1001 only

ACCOUNT in CATEGORY 1001 is editable

TEMENOS ACCOUNT,CAT1001 | 19461

Current Account 19461 WILLIAM HEWLETT

Customer ID	100366	William Hewlett
Product Code *	1001	Current Account
Currency	USD	US Dollar
Mnemonic	HEWWILLUSD	
Account Name 1	EN WILLIAM HEWLETT	
Account Name 2	EN	

Practice – Version Hook for ID

- Create a VERSION for the CUSTOMER that prefixes “99” to the ID given by the user
- Expected Output

The screenshot displays the TEMENOS user interface for managing customer information. At the top, a blue header bar contains the TEMENOS logo and a search bar with the text "CUSTOMER, ID99 | 77". Below the header, the "Basic Details" section is highlighted, showing the ID "9977". The form includes fields for "Title", "Given Name", and "Family Name", each with a dropdown menu. Below these, there are three rows for "Full Name", "Full Name-2", and "Short Name", each with a dropdown menu and a text input field. The dropdown menus are currently set to "EN".

Field	Value
Title	- Please Select
Given Name	
Family Name	
Full Name *	EN
Full Name-2	EN
Short Name *	EN

Lesson 3. ENQUIRY Hooks in Java

T24 Extensibility in Java

| Enquiry hooks

We can customize ENQUIRY by

- modifying the dynamic selection box – BUILD hook

- modifying the data before display – CONVERSION hook

Extend the **com.temenos.t24.api.hook.system.Enquiry** class

- setRecord()

- setFilterCriteria()

- setValue()

- setIds()

| DEMO – Enquiry Build hook

- The Build hook in ENQUIRY, is used to modify the selection criteria
- We can set the Field Name, Operator and the value and add these to the filter criteria

Create an enquiry to display Customer Id, Account Id and the respective working balance after accepting category from the user.

Based on category, additional filtering has to be done

- For category 1001 - Balance should be in the range 0 and 49999
- For category 6001 - Balance should be in the range 50000 and 100000
- For category greater than 7000 - Balance should be in the range 100000 and 500000

Step 1 – Create the class

New Java Class
Create a new Java class.

Source folder: T24EXTEN/src Browse...

Package: com.newbank Browse...

☐ Enclosing type: Browse...

Name: ACEnquiryDemo

Modifiers: ☒ public ☐ package ☐ private ☐ protected
☐ abstract ☐ final ☐ static

Superclass: java.lang.Object Browse...

Interfaces: Add... Remove

Which method stubs would you like to create?
☐ public static void main(String[] args)
☐ Constructors from superclass
☒ Inherited abstract methods

Do you want to add comments? (Configure templates and default value [here](#))
☒ Generate comments

Finish Cancel

Superclass Selection

Choose a type: com.temenos.t24.api.system.e

Matching items: Enquiry - com.temenos.t24.api.hook.system

com.temenos.t24.api.hook.system - C:\L3API\Tem...com\temenos\t24\main\t24lib\EB_EnquiryHook...

OK Cancel

```
package com.newbank;

import com.temenos.t24.api.hook.system.Enquiry;

/**
 * demo for Build routine
 */
public class ACEnquiryDemo extends Enquiry {

}
```

Override/Implement Methods

Select methods to override or implement:

- ☒ Enquiry
 - ☒ setFilterCriteria(String, List<FilterCriteria>)
 - ☐ setIds(List<FilterCriteria>)
 - ☐ setRecord(String, String, TStructure, List<FilterCriteria>)
 - ☐ setValue(String, String, TStructure, List<FilterCriteria>)
- ☐ T24Context
- ☐ Object

Insertion point: First member

☒ Generate method comments

The format of the method stubs may be configured on the [Code Templates](#) preference page.

1 of 19 selected.

OK Cancel

Step 1 – Create the class

```
package com.newbank;

import java.util.List;

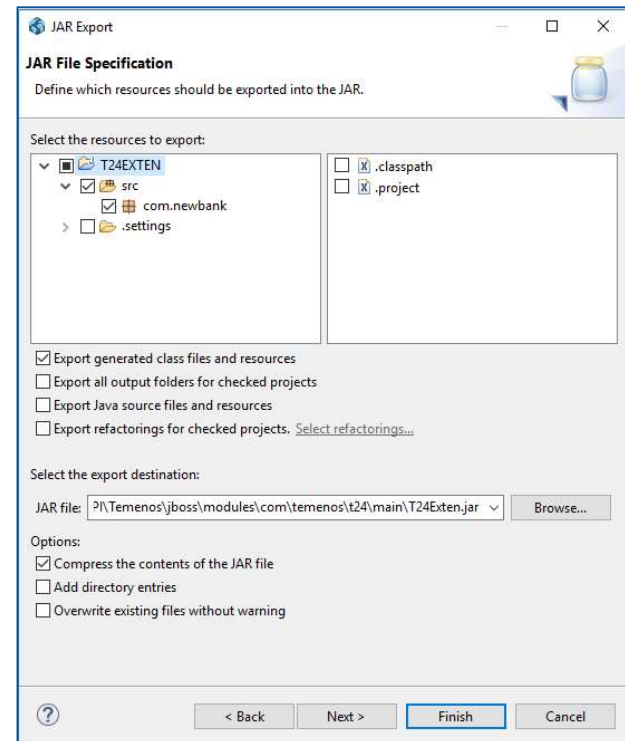
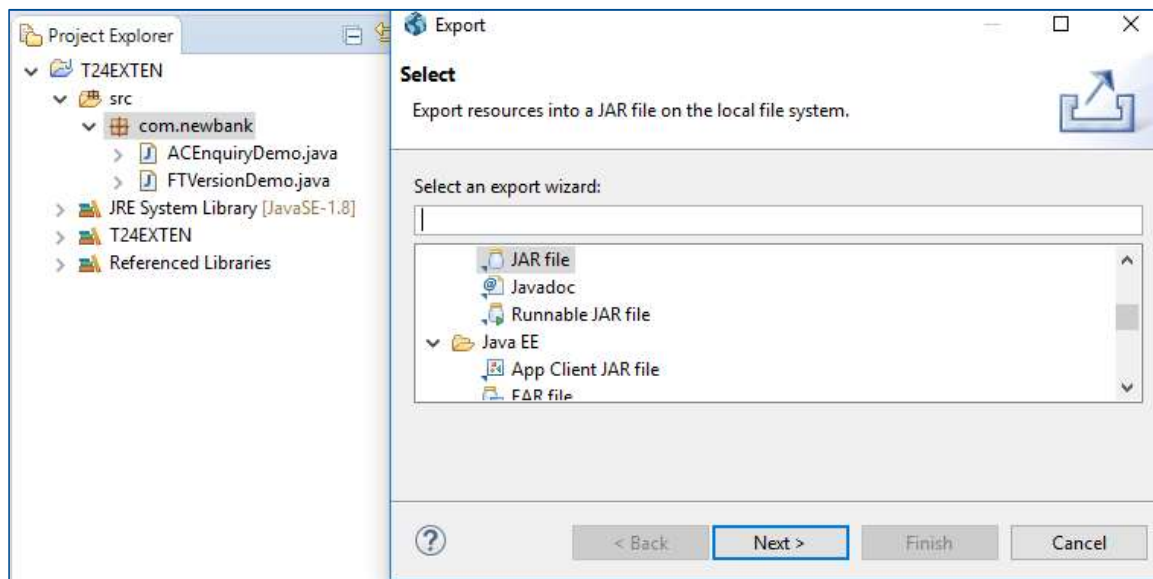
import com.temenos.t24.api.complex.eb.enquiryhook.FilterCriteria;
import com.temenos.t24.api.hook.system.Enquiry;

/**
 * demo for ENQUIRY Build hook
 */
public class ACEnquiryDemo extends Enquiry {

    @Override
    public List<FilterCriteria> setFilterCriteria(String enquiryName, List<FilterCriteria> filterCriteria) {
        FilterCriteria fc = new FilterCriteria();
        String s1 = filterCriteria.get(0).getFieldname();
        String s3 = filterCriteria.get(0).getValue();

        fc.setFieldname("WORKING.BALANCE");
        fc.setOperand("RG");
        if (s1.equals("CATEGORY")) {
            if (Integer.parseInt(s3) > 7000) {
                fc.setValue("50000 100000");
            } else {
                switch (s3) {
                    case "1001":
                        fc.setValue("0 10000");
                        break;
                    case "6001":
                        fc.setValue("10000 50000");
                        break;
                    default:
                }
            }
        }
        filterCriteria.add(fc);
        return filterCriteria;
    }
}
```

Step 2 – Create JAR and update module.xml



Name	Date modified	Type	Size
t24lib	1/30/2019 7:44 AM	File folder	
module	2/18/2019 6:31 PM	XML Document	71 KB
T24Ext.jar	2/18/2019 10:03 PM	Executable Jar File	3 KB

```
1 <module xmlns="urn:jboss:module:1.0" name="com.temenos.t24">
2   <resources>
3     <!-- Insert resources here -->
4     <resource-root path="./T24Ext.jar" />
5     <resource-root path="./t24lib/AA_ARAccountsData.jar" />
6     <resource-root path="./t24lib/AA_ARC.jar" />
```

Step 3 – EB.API

```
package com.newbank;

import java.util.List;

import com.temenos.t24.api.complex.eb.enquiryhook.FilterCriteria;
import com.temenos.t24.api.hook.system.Enquiry;

/**
 * demo for ENQUIRY Build hook
 */
public class ACEnquiryDemo extends Enquiry {

    @Override
    public List<FilterCriteria> setFilterCriteria(String enquiryName, List<FilterCriteria> filterCriteria) {
        FilterCriteria fc = new FilterCriteria();
        String s1 = filterCriteria.get(0).getFieldname();
        String s3 = filterCriteria.get(0).getValue();

        fc.setFieldname("WORKING.BALANCE");
        fc.setOperand("RG");
        if (s1.equals("CATEGORY")) {
            if (Integer.parseInt(s3) > 7000) {
                fc.setValue("50000 100000");
            } else {
                switch (s3) {
                    case "1001":
                        fc.setValue("0 10000");
                        break;
                    case "6001":
                        fc.setValue("10000 50000");
                        break;
                    default:
                }
            }
        }
        filterCriteria.add(fc);
        return filterCriteria;
    }
}
```

The screenshot shows the TEMENOS EB.API ACENQ.DEMO configuration window. The interface includes a header with the TEMENOS logo and the title 'EB.API | ACENQ.DEMO'. Below the header, there are tabs for 'EB.API' and 'ACENQ.DEMO'. The main configuration area contains several fields and options:

- Description:** EN Demo for Conversion routine
- Protection Level *:** Radio buttons for Full (selected), Partial, and None.
- Source Type *:** Radio buttons for Basic, Java, Hook, and Method (selected).
- Java Method:** A text field containing 'setFilterCriteria'.
- Java Class:** A text field containing 'ACEnquiryDemo'.
- Java Package:** A text field containing 'com.newbank'.

Each of the last three fields (Java Method, Java Class, and Java Package) is highlighted with a yellow border in the original image.

Step 4 – Create ENQUIRY and attach hook

The screenshot shows the TEMENOS ENQUIRY,DS I ACENQ configuration interface. The header bar includes the TEMENOS logo and the text "ENQUIRY,DS I ACENQ". Below the header, the "Enquiry Ds" section is active, showing "ACENQ". The main configuration area contains several fields and options:

- Sort.1**: WORKING.BALANCE DSND
- Open Bracket.1**: ☐ Yes
- Selection Flds.1**: CATEGORY
- Sel Label.1**: EN
- Sel Fld Oper.1**: (empty)
- Required Sel.1**: ☐
- Close Bracket.1**: ☐ Yes
- Rel Next Field.1**: ☐ Or ☐ And
- Build Routine.1**: ACENQ.DEMO (highlighted with a yellow box)
- Header**: @ (1,1) Build routine Demo

Launch the ENQUIRY

TEMENOS ENQ ACENQ CATEGORY EQ 1001

Build routine Demo

Account Number	Category	Ccy	Working Balance
74918	1001	USD	10,000.00
76341	1001	EUR	10,000.00
77868	1001	GBP	10,000.00
78007	1001	USD	10,000.00
81884	1001	CAD	10,000.00
83666	1001	USD	10,000.00
76945	1001	USD	9,960.00
78883	1001	GBP	9,420.25
81744	1001	EUR	9,000.00
76748	1001	USD	8,013.26
77445	1001	USD	8,000.00
78581	1001	USD	8,000.00
81752	1001	USD	8,000.00
74567	1001	GBP	7,658.74
74586	1001	GBP	7,658.74
78794	1001	GBP	7,658.74
74578	1001	GBP	7,558.74
78042	1001	USD	7,500.00
78514	1001	USD	7,000.00
77698	1001	GBP	6,000.00
34533	1001	USD	5,300.00
75515	1001	USD	5,040.00
81768	1001	GBP	4,500.00
83224	1001	USD	4,500.00

TEMENOS ENQ ACENQ CATEGORY EQ 6001

Build routine Demo

Account Number	Category	Ccy	Working Balance
82384	6001	USD	50,000.00
78937	6001	USD	46,245.00
82414	6001	USD	45,000.00
78905	6001	USD	44,000.00
78867	6001	USD	20,885.00
74381	6001	USD	18,000.00
76422	6001	USD	17,194.73
78719	6001	USD	15,650.00
74365	6001	USD	15,000.00
17531	6001	EUR	14,753.43
78875	6001	USD	13,000.00
78751	6001	USD	12,000.00
78573	6001	GBP	10,482.55

TEMENOS ENQ ACENQ CATEGORY GT 7001

Build routine Demo

Account Number	Category	Ccy	Working Balance
GBPUSD1902201010001	19022	GBP	100,000.00
USD1661100040001	16611	USD	95,000.00
USD1422000060001	14220	USD	94,298.93
USD1660400060001	16604	USD	78,750.00
EUR1419700010001	14197	EUR	67,377.08
EUR1401000010001	14010	EUR	59,155.44
USD1660400050001	16604	USD	55,150.00

Practice – Enquiry Conversion hook

- Create an ENQUIRY TO LIST the LD records that are in foreign currency. List the ID, CUSTOMER, AMOUNT and Equivalent amount in LCCY.
- Use conversion hook to convert the amount in foreign currency to LCCY
- Expected output

CURRENCY	JPY	Japanese Yen
Mid Reval Rate.1	110.624307	

Conversion routine demo					
Contract Number	Product	Customer Id	Currency	Loan Amount	Loan Amount
LD1808014199	21003	190002	JPY	75,000.00	677.97
LD1809452323	21010	190114	JPY	40,000,000.00	361,584.19
LD1809497668	21074	190114	JPY	4,000,000.00	36,158.42

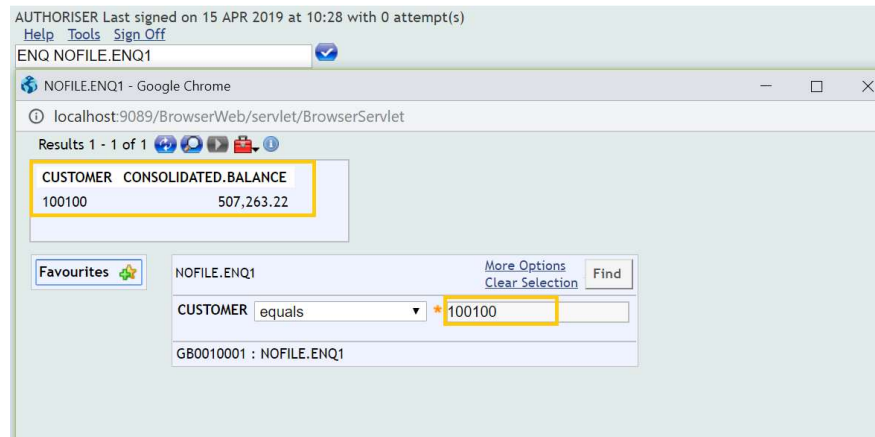
Calculator	Standard
677.9703487769645	
75000 ÷ 110.624307 = 677.9703487769645	

| NOFILE Enquiry

- What if there are multiple files involved in creation of an enquiry?
- What if the record ID of another application is not part of the base application?
- This can be achieved using NOFILE enquiries
- Since more than one file might be required to extract data, it is a NOFILE enquiry. We can extract data from any number of T24 applications
- Subroutine is written to extract all required data and form the record
- **ENQUIRY** application is used to display extracted data

DEMO – NOFILE enquiry

- Create an ENQUIRY to display the consolidated balance for a CUSTOMER.
- Hint : Accumulate all the balance from all the accounts available in CUSTOMER.ACCOUNT.
- Expected Output



The screenshot shows a web application interface for an enquiry. At the top, it says 'AUTHORISER Last signed on 15 APR 2019 at 10:28 with 0 attempt(s)' with links for 'Help', 'Tools', and 'Sign Off'. Below this is a search bar containing 'ENQ NOFILE.ENQ1'. The browser title is 'NOFILE.ENQ1 - Google Chrome' and the address bar shows 'localhost:9089/BrowserWeb/servlet/BrowserServlet'. The main content area displays 'Results 1 - 1 of 1' and a table with two columns: 'CUSTOMER' and 'CONSOLIDATED.BALANCE'. The table has one row with the values '100100' and '507,263.22'. Below the table, there is a 'Favourites' button and a search section. The search section includes a dropdown menu set to 'CUSTOMER equals', a text input field containing '100100', and buttons for 'More Options', 'Clear Selection', and 'Find'. At the bottom, it shows 'GB0010001 : NOFILE.ENQ1'.

CUSTOMER	CONSOLIDATED.BALANCE
100100	507,263.22

Step 1 – Create the class

Source folder: T24EXT/src Browse...

Package: com.newbank Browse...

☐ Enclosing type: Browse...

Name: NoFileEnquiryDemo

Modifiers: ☒ public ☐ package ☐ private ☐ protected
☐ abstract ☐ final ☐ static

Superclass: java.lang.Object

Interfaces:

Which method stubs would you like to create?

☐ public static void main(String[] args)
☐ Constructors from superclass
☒ Inherited abstract methods

Do you want to add comments? (Configure templates and default value [here](#))
☒ Generate comments

Superclass Selection

Choose a type: com.temenos.t24.api.hook.system.e

Matching items:

Enquiry - com.temenos.t24.api.hook.system

com.temenos.t24.api.hook.syste...main\t24lib\EB_EnquiryHook.jar

OK Cancel

InvocationTargetException.class Check.java NoFileEnquiryDemo.java

```
@Override
public List<String> setIds(List<FilterCriteria> filterCriteria) {
    Customer cusrec = new Customer(this);
    DataAccess da = new DataAccess(this);
    String s1 = filterCriteria.get(0).getFieldname();
    String s3 = filterCriteria.get(0).getValue();
    List<String> retId = new ArrayList<String>();
```

Step 1 – Create the class

```
NoFileEnquiryDemo.java
package com.newbank;

import java.util.ArrayList;

public class NoFileEnquiryDemo extends Enquiry {

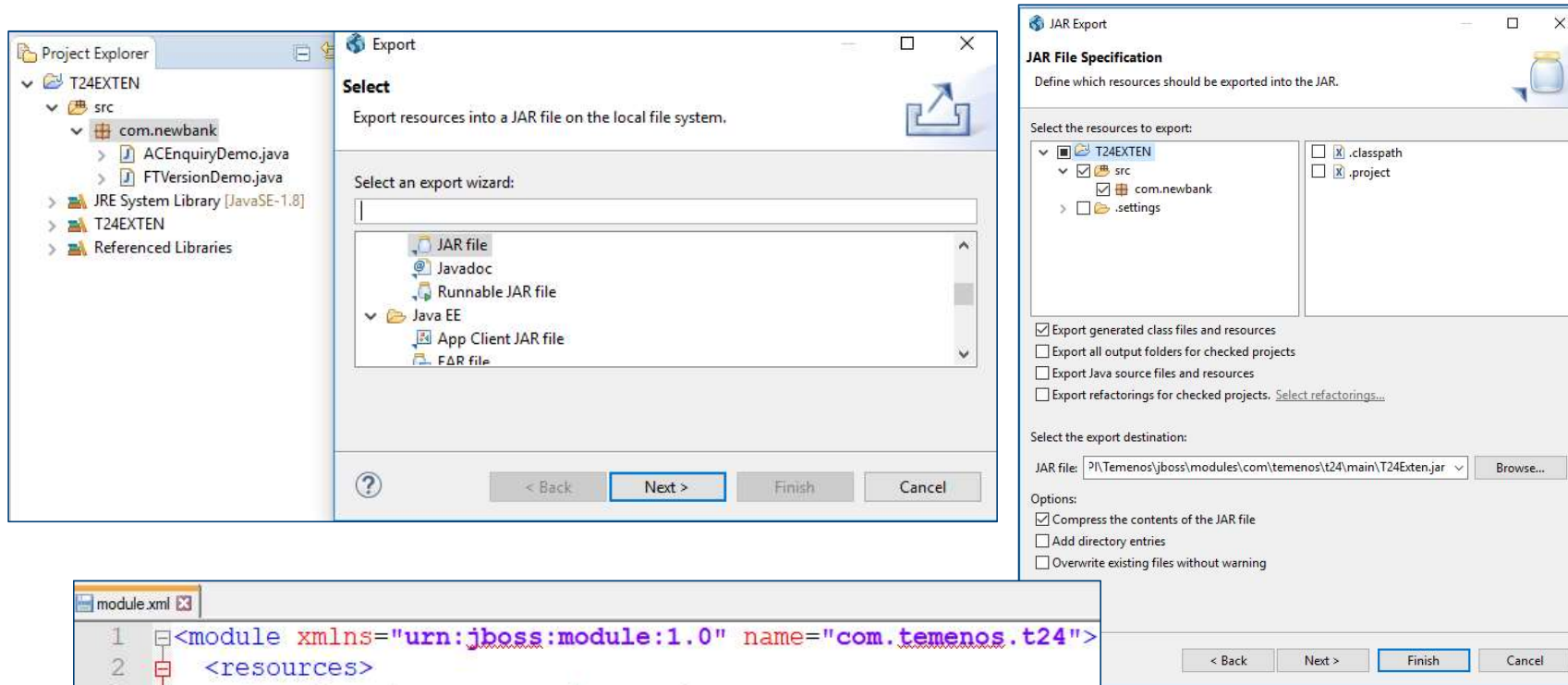
    @Override
    public void setRecord(String value, String currentId, TStructure currentRecord,
        List<FilterCriteria> filterCriteria) {
    }

    @Override
    public List<FilterCriteria> setFilterCriteria(String enquiryName, List<FilterCriteria> filterCriteria) {
        // TODO Auto-generated method stub
        return null;
    }

    @Override
    public String setValue(String value, String currentId, TStructure currentRecord,
        List<FilterCriteria> filterCriteria) {
        // TODO Auto-generated method stub
        return null;
    }

    public List<String> setIds(List<FilterCriteria> filterCriteria) {
        Customer cusrec = new Customer(this);
        DataAccess da = new DataAccess(this);
        String s1 = filterCriteria.get(0).getFieldname();
        String s3 = filterCriteria.get(0).getValue();
        List<String> retId = new ArrayList<String>();
        Double consAmt = 0.0;
        if (s1.equals("CUSTOMER")) {
            if (!s3.equals("")) {
                cusrec.setCustomerId(s3);
                List<String> accnum = null;
                try {
                    accnum = cusrec.getAccountNumbers();
                    for (String acnum : accnum) {
                        AccountRecord ar = new AccountRecord(da.getRecord("ACCOUNT", acnum));
                        String amt = ar.getWorkingBalance().getValue();
                        if (amt.isEmpty() || amt.equals("")) {
                            continue;
                        } else {
                            double amt1 = Double.valueOf(amt);
                            consAmt += amt1;
                        }
                    }
                } catch (Exception e) {
                }
                retId.add(Double.toString(consAmt));
            }
        }
        return retId;
    }
}
```

Step 2 – Create JAR and update module.xml

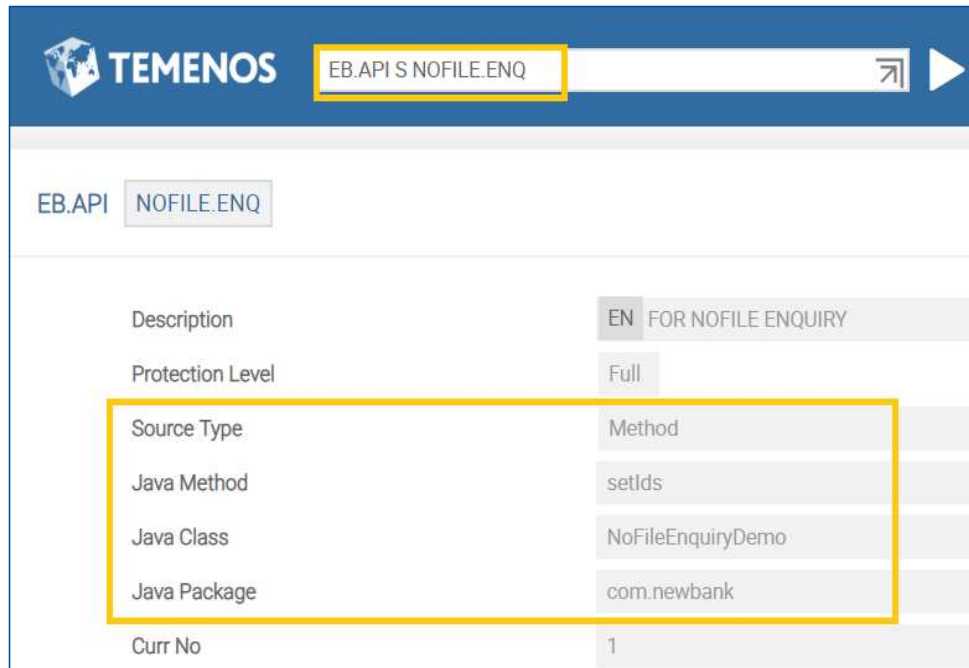


The screenshot shows the Eclipse IDE interface. On the left, the Project Explorer displays the project structure: T24EXTEN, src, com.newbank, ACEnquiryDemo.java, FTVersionDemo.java, JRE System Library [JavaSE-1.8], T24EXTEN, and Referenced Libraries. The main editor area shows the 'Export' wizard. The 'Select' step is active, showing 'Export resources into a JAR file on the local file system.' The 'Select an export wizard:' list has 'JAR file' selected. The 'JAR File Specification' dialog is open, showing 'Select the resources to export:' with 'T24EXTEN' selected, and 'src' and 'com.newbank' checked. The 'Export generated class files and resources' checkbox is checked. The 'JAR file:' field shows the path 'P:\Temenos\jboss\modules\com\temenos\t24\main\T24Exten.jar'. The 'Options' section has 'Compress the contents of the JAR file' checked. The 'Finish' button is highlighted.

module.xml

```
1 <module xmlns="urn:jboss:module:1.0" name="com.temenos.t24">
2   <resources>
3     <!-- Insert resources here -->
4     <resource-root path="./T24Exten.jar" />
5     <resource-root path="./t24lib/AA_ARAccountsData.jar" />
6     <resource-root path="./t24lib/AA_ARC.jar" />
```

Step 3 – EB.API



Description	EN FOR NOFILE ENQUIRY
Protection Level	Full
Source Type	Method
Java Method	setIds
Java Class	NoFileEnquiryDemo
Java Package	com.newbank
Curr No	1

| Creating the **ENQUIRY** Record

- A valid application name should be given in the field FILE.NAME which is checked against **STANDARD.SELECTION**
- A NOFILE enquiry does not have a single associated file
- Create a record in **STANDARD.SELECTION** with **NOFILE** prefix for the ID e.g. NOFILE.CUSBALANCE
- This ID should be given in FILE.NAME in the **ENQUIRY** application

NOFILE Enquiry - STANDARD.SELECTION

The screenshot shows the TEMENOS interface for the NOFILE Enquiry - STANDARD.SELECTION. The header bar includes the TEMENOS logo and a search bar containing 'SS S NOFILE.DEMO.NEW'. Below the header, the title 'STANDARD.SELECTION' and the user 'NOFILE.DEMO.NEW' are displayed. The main area contains a table of user-defined fields. Annotations highlight specific fields: 'Selection Field' points to 'Usr Type.1' with value 'S'; 'Field created by the routine' points to 'Usr Field Name.2' with value 'CONSOLIDATED.BALANCE' and 'Usr Field No.2.1' with value 'NOFILE.ENQ'; and 'ID of the EB.API' points to the value 'NOFILE.ENQ'.

Field Name	Value
Usr Field Name.1	CUSTOMER
Usr Type.1	S
Usr Display Fmt.1	10L
Usr Single Mult.1	S
Usr Field Name.2	CONSOLIDATED.BALANCE
Usr Type.2	R
Usr Field No.2.1	NOFILE.ENQ
Usr Display Fmt.2	100L
Usr Single Mult.2	S

NoFile Enquiry - ENQUIRY

ENQUIRY NOFILE.DEMO

Page Size	4,20
File Name	NOFILE.DEMO
Fixed Selection.1	CONSOLIDATED.BAL NE *
Selection Flds.1	CUSTOMER
Required Sel.1	Y
Field Name.1	CUSTOMER
Operation.1 .1	SELECTION CUSTOMER
Column.1	1
Length Mask.1	10L
Single Multi.1	S
Field Name.2	CONSOLIDATED.BAL
Operation.2 .1	0

ENQUIRY NOFILE.DEMO

Field Name.2	CONSOLIDATED.BAL
Operation.2 .1	0
Column.2	2
Conversion.2 .1	F *,2,1
Page Fields	2
Multi Fields	0
Break Fields	0
Process Breaks	0
Total Fields	0
Curr No	13
Inputter.1	202_INPUTTER_OFS_BROWSERTC
Date time.1	07 AUG 19 13:41
Authoriser	202_INPUTTER_OFS_BROWSERTC

NoFile Enquiry – Execution

INPUTTER Last signed on 27 MAR 2019 at 16:31 with 0 attempt(s)
[Help](#) [Tools](#) [Sign Off](#)

ENQ NOFILE.ENQ1 

NOFILE.ENQ1 - Google Chrome

localhost:9089/BrowserWeb/servlet/BrowserServlet

Favourites 

NOFILE.ENQ1 [More Options](#) [Clear Selection](#) **Find**

CUSTOMER	equals ▼	 100100
NO.SORT.OPTION	equals ▼	
INCLUDE.DL	equals ▼	

GB0010001 : NOFILE.ENQ1

NoFile Enquiry – OUTPUT

AUTHORISER Last signed on 15 APR 2019 at 10:28 with 0 attempt(s)
[Help](#) [Tools](#) [Sign Off](#)

ENQ NOFILE.ENQ1 

NOFILE.ENQ1 - Google Chrome

localhost:9089/BrowserWeb/servlet/BrowserServlet

Results 1 - 1 of 1     

CUSTOMER	CONSOLIDATED.BALANCE
100100	507,263.22

Favourites 

NOFILE.ENQ1 [More Options](#) [Clear Selection](#) [Find](#)

CUSTOMER equals  * 100100

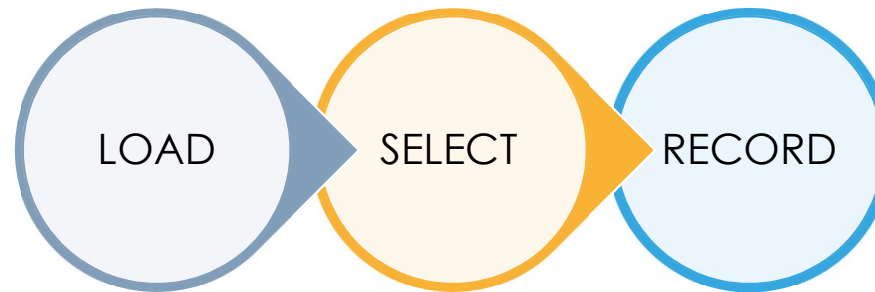
GB0010001 : NOFILE.ENQ1

Lesson 4. SERVICE Hooks in Java

T24 Extensibility in Java

| Services in T24

- Multithreaded processes
- Can be run independently or as part of COB



- Must implement **`com.temenos.t24.api.hook.system.ServiceLifecycle`**
- Equivalent of
 - LOAD – `initialize()`
 - SELECT – `getIds()` or `getTableName()`
 - RECORD – `inputRecord()` or `process()`

DEMO- Service

- Simple service that updates the SHORT.NAME of CUSTOMERS in SECTOR 1001 as TEST + existing MNEMONIC
- EXPECTED OUTPUT

CUSTOMER 130131	
Customer Mnemonic	KATRIN
Short Name	EN TEST KATRIN
Full Name	EN Abbey Adalwine
Street Name	EN FRANKFURT

1. Create the class

```
package com.newbank;

import java.util.List;

import com.temenos.api.TBoolean;
import com.temenos.api.TStructure;
import com.temenos.t24.api.complex.eb.servicehook.ServiceData;
import com.temenos.t24.api.hook.system.ServiceLifecycle;
import com.temenos.t24.api.records.customer.CustomerRecord;
import com.temenos.t24.api.system.DataAccess;

/* Service DEMO */

public class MyServiceDemo extends ServiceLifecycle {
    @Override
    public void inputRecord(String id, ServiceData serviceData, String controllItem, TBoolean setZeroAuth,
        List<String> versionNames, List<String> recordIds, List<TStructure> records) {

        DataAccess da= new DataAccess(this);
        versionNames.add("CUSTOMER");
        recordIds.add(id);
        CustomerRecord cusRec = new CustomerRecord(da.getRecord("CUSTOMER", id));
        cusRec.setShortName("TEST "+cusRec.getMnemonic().toString(), 0);
        records.add(cusRec.toStructure());
        setZeroAuth.set(true);
    }

    @Override
    public List<String> getIds(ServiceData serviceData, List<String> controllList) {

        List<String> recIds = null;
        DataAccess da = new DataAccess(this);
        recIds=da.selectRecords("BNK", "CUSTOMER", "", "WITH MNEMONIC EQ KATRIN KMART KOS ");
        return recIds;
    }
}
```

Export JAR and
restart jBoss

2. Create the EB.API records

```
public class MyServiceDemo extends ServiceLifecycle {  
    @Override  
    public void inputRecord(String id, ServiceData serviceData,  
        List<String> versionNames, List<String> recordIds,  
  
        DataAccess da= new DataAccess(this);  
        versionNames.add("CUSTOMER");  
        recordIds.add(id);  
        CustomerRecord cusRec = new CustomerRecord(da.getRecord  
        cusRec.setShortName("TEST "+cusRec.getMnemonic().toStri  
        records.add(cusRec.toStructure());  
        setZeroAuth.set(true);  
  
    }  
    @Override  
    public List<String> getIds(ServiceData serviceData, List<St  
  
        List<String> recIds = null;  
        DataAccess da = new DataAccess(this);  
        recIds=da.selectRecords("BNK", "CUSTOMER", "", "WITH MN  
        return recIds;  
  
    }  
}
```

EB.API SERVICE.DEMO	
Description	EN SERVICE DEMO
Protection Level	Full
Source Type	Method
Java Method	inputRecord
Java Class	MyServiceDemo
Java Package	com.newbank

EB.API SERVICE.DEMO.SELECT	
Description	EN SERVICE DEMO SELECT
Protection Level	Full
Source Type	Method
Java Method	getIds
Java Class	MyServiceDemo
Java Package	com.newbank

3. Create the PGM.FILE, BATCH & TSA.SERVICE records

PGM.FILE SERVICE.DEMO 

Type	B
Batch Job.1	@BATCH.JOB.CONTROL
Product	EB

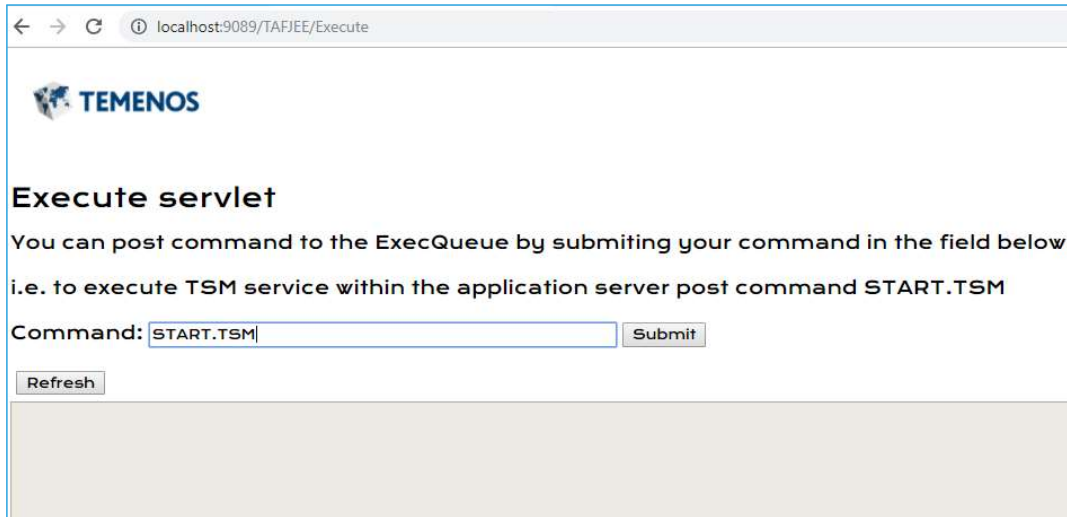
BATCH BNK/SERVICE.DEMO

Process Status	O
Batch Environment	F FOREGROUND
Job Name.1	SERVICE.DEMO
Frequency.1	D
Data.1.1	BULK.OFS

TSA.SERVICE BNK/SERVICE.DEMO

Description.1	SERVICE DEMO 
	
Server Name.1	
Work Profile.1	
Server Status.1	
User *	INPUTTER INPUTTER 
Service Control	☐ Stop ☒ Start ☐ Auto

4. Run the TSM and TSA



← → ↻ ⓘ localhost:9089/TAFJEE/Execute

 **TEMENOS**

Execute servlet

You can post command to the ExecQueue by submitting your command in the field below.
i.e. to execute TSM service within the application server post command START.TSM

Command:

- When the service is run, we can see the SHORT.NAME is updated



Customer No	Mnemonic	Name	Account Officer
<u>130131</u>	KATRIN	TEST KATRIN	Implementation
<u>100284</u>	KMART	TEST KMART	Branch Operations Manager
<u>100329</u>	KOS	TEST KOS	Derivatives Front Office Dealer

| Lesson 5. Local APPLICATION

T24 Extensibility in Java



TEMENOS
Learning Community

DEMO Creating Local Application

- We can create a new application using EB.TABLE.DEFINITION. OVERRIDE must be raised if the phone / email is used by existing customer

TABLE.NAME	WALKIN.CUSTOMER
PRODUCT	ST
FILE.TYPE	H
CLASSIFICATION	CUS

@ID, OVERRIDE, AUDIT
fields are added
automatically

Field Name	Data Type	Remarks
NAME	A	
PHONE		Raise override if this PHONE is available in CUSTOMER
EMAIL		Raise override if this EMAIL is available in CUSTOMER

1. Creating the TABLE – EB.TABLE.DEFINITION

- Create the table using EB.TABLE.DEFINITION
- ETD creates the required artefacts – PGM.FILE, FILE.CONTROL, STANDARD.SELECTION, DATA FILES

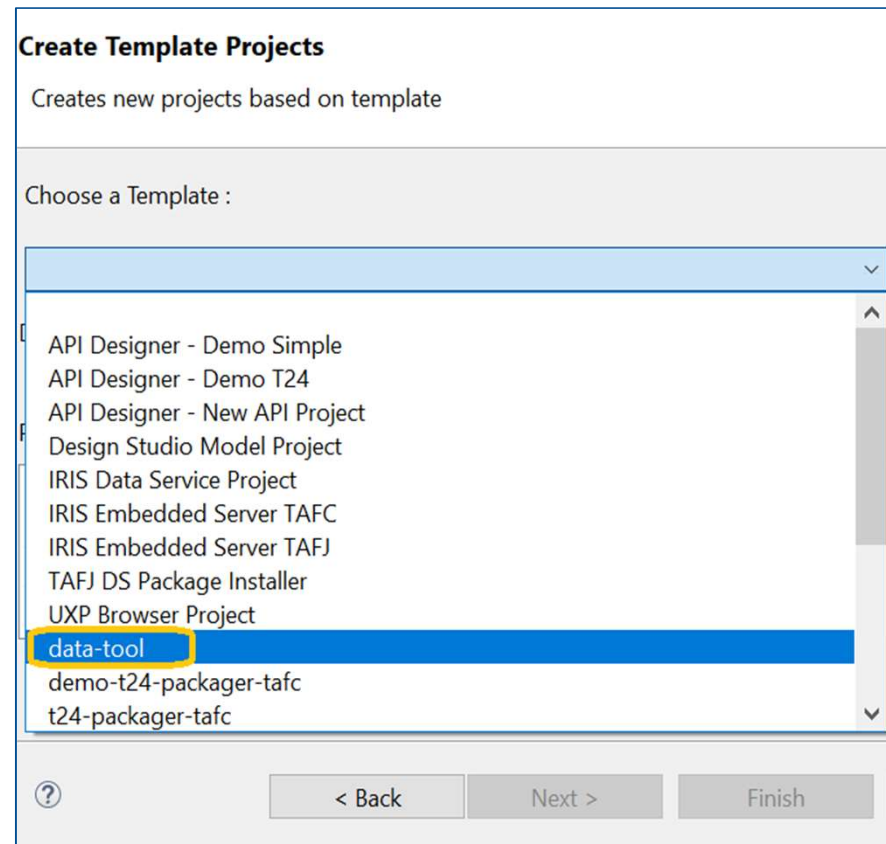
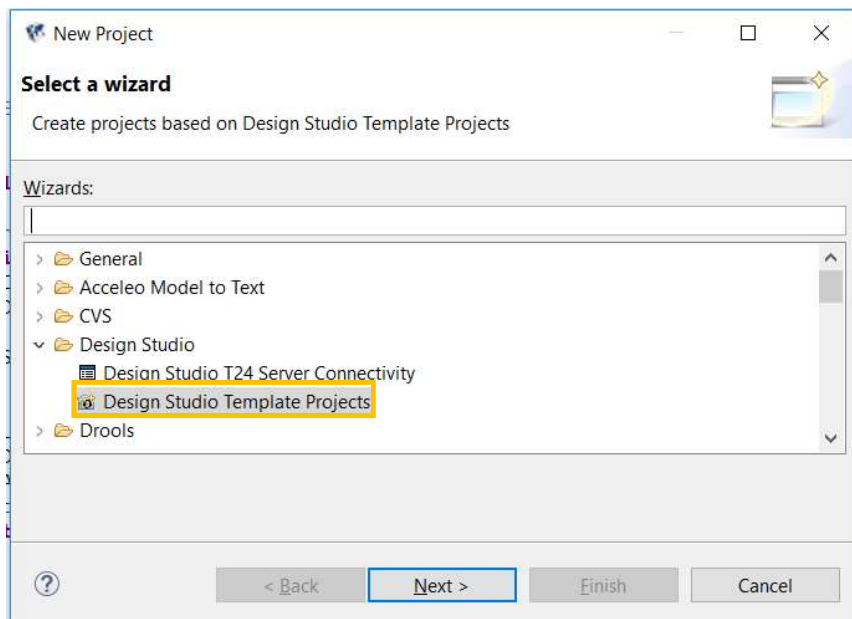
EB.TABLE.DEFINITION		WALKIN.CUSTOMER
Description	EN WALKIN CUSTOMER DEMO	
Product	ST	System Tables
Field Name.1	NAME	
Description.1	NAME.OF.CUST	
Max Char.1	35	
Data Type.1	ALPHANUMERIC	Alpha Numeric Charactes
Field Name.2	PHONE	
Description.2	PHONE	
Max Char.2	20	
Data Type.2	ALPHANUMERIC	Alpha Numeric Charactes
Field Name.3	EMAIL	
Description.3	EMAIL	
Max Char.3	35	
Data Type.3	ALPHANUMERIC	Alpha Numeric Charactes
Prefix	ST.WAL26	
File Type	Cus	
Link To Wfl	No	

| 2. Generate the API for the new Table

- Using Design Studio, we can generate the API for the APPLICATION created using EB.TABLE.DEFINITION
- we need a server project – to establish connection with T24
- We need a models project – to import the application from T24

2a. Create the –DataTool project

- Creating the server project
- File→New→Project



2b. Create the DS-DataTool Project

- Create the DS-DataTool project

Create Template Projects
Creates new projects based on template

Choose a Template : Reload...

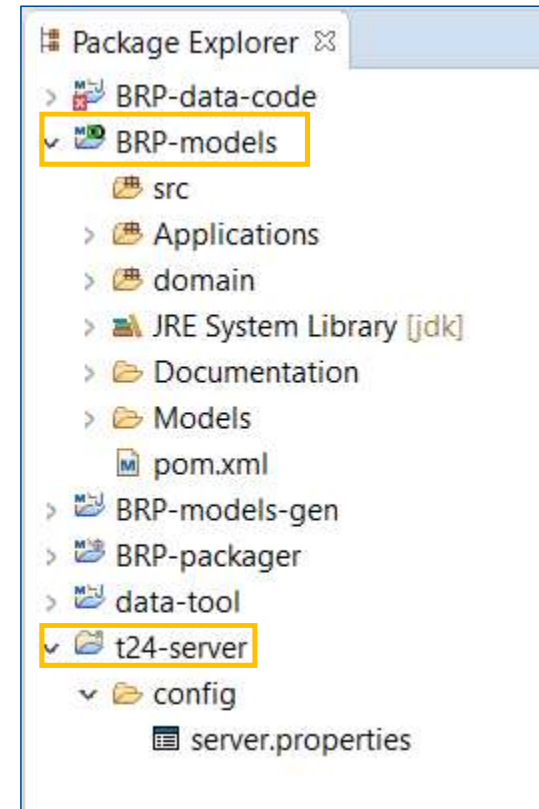
data-tool 201906.0.0-SNAPSHOT

Description :
T24 Data Tool (EXPERIMENTAL)

Parameters :

Name	Value	Description
project-name	BRP	Name of the project
product-name	XX	Product's name
component-name	Sample	Component's name
host	localhost	Host
ws-port	9089	Port
t24username	INPUTT	User name
t24password	123456	Password
t24username-authorizer	AUTHOR	Authoriser's user name
t24password-authorizer	123456	Authoriser's password
tafjHome	C:\Temenos\TAFJ	TAFJ home (tafj.home in tafj.prope
insertDir	C:\Temenos\t24home\default\RG.BP	Insert BASIC sources directory (tem
libDir	C:\Temenos\jboss\modules\com\temenos\t24\main	Precompile classes directory (temn

< Back Next > Finish Cancel



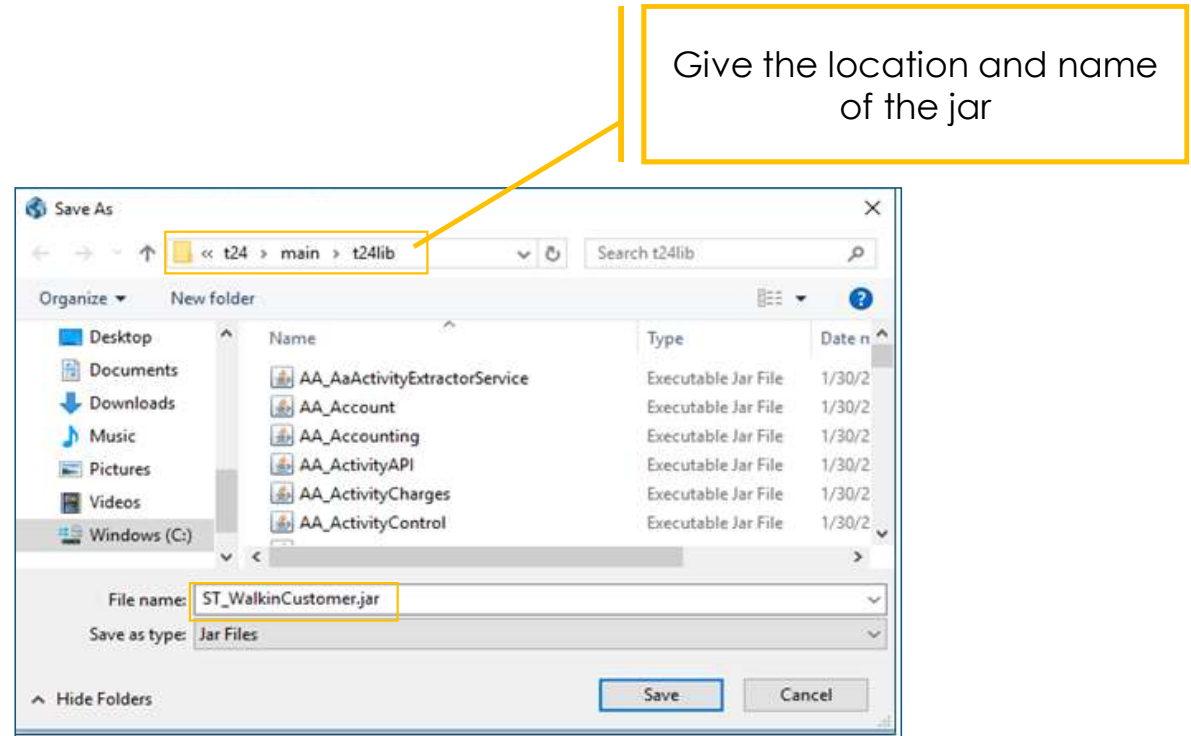
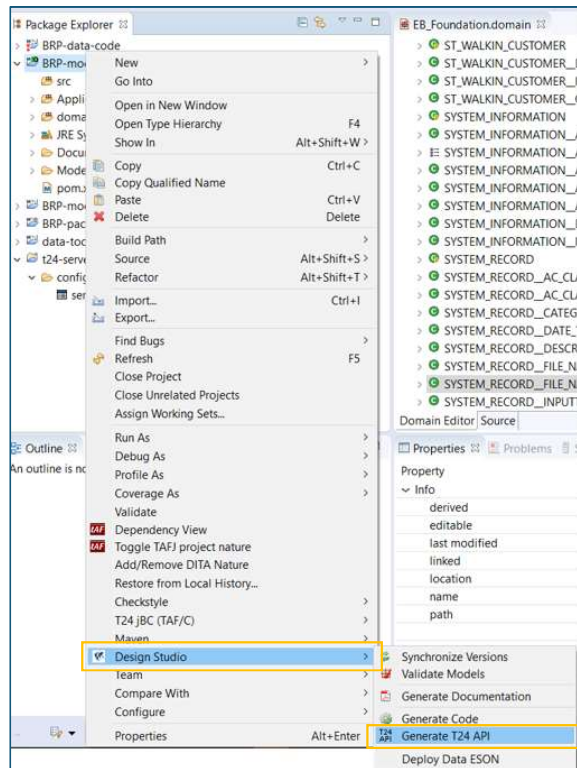
2c. Import the Local Application

The screenshot illustrates the process of importing a local application into the T24 Applications environment using the Eclipse IDE. The process involves several steps and dialog boxes:

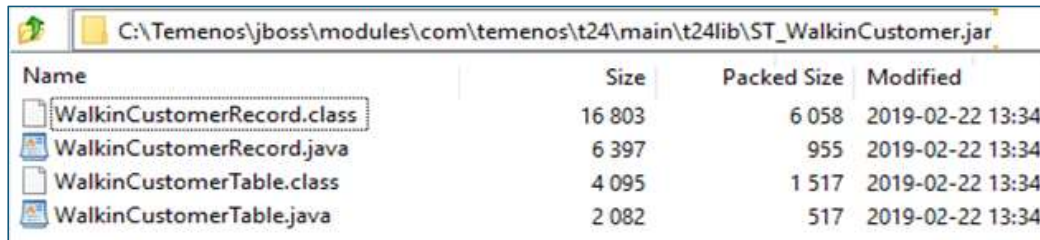
- Package Explorer:** The left sidebar shows the project structure. The **BRP-models** folder is expanded, and the **Import...** option is selected from the context menu.
- Import Dialog:** The 'Import' dialog box is open, showing 'Import T24 Applications By Name' as the selected option under the 'Design Studio' category.
- T24 Applications Import Wizard:** The wizard is open, showing the 'T24 Application Name Selection' step. The 'Application Name' is set to **ST.WALKIN.CUSTOMER**.
- Select the destination of imported Applic:** The wizard prompts to select the destination folder. The **Applications** folder under **BRP-models** is selected.
- Projects visibility:** The 'Only model projects' radio button is selected.
- Folder Structure:** The 'FLAT' option is selected for the folder structure.
- EB_Foundation.domain:** The right sidebar shows the contents of the **EB_Foundation.domain** project. The **ST_WALKIN_CUSTOMER** application is highlighted, along with its associated routines and values.

2d. Generate the API for the new Table

- Toggle the DS project as a TAFJ project
- Right click on project → Design Studio → Generate T24 API



2e. Update module.xml and Build path

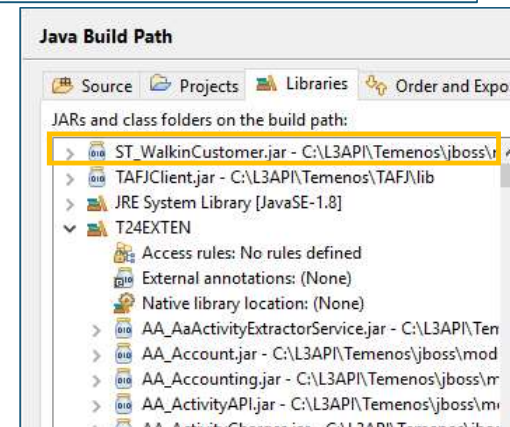


Name	Size	Packed Size	Modified
WalkinCustomerRecord.class	16 803	6 058	2019-02-22 13:34
WalkinCustomerRecord.java	6 397	955	2019-02-22 13:34
WalkinCustomerTable.class	4 095	1 517	2019-02-22 13:34
WalkinCustomerTable.java	2 082	517	2019-02-22 13:34

- Update module.xml and restart jBoss

```
<module xmlns="urn:jboss:module:1.0" name="com.temenos.t24">
  <resources>
    <!-- Insert resources here -->
    <resource-root path="./T24Exten.jar" />
    <resource-root path="./t24lib/ST_WalkinCustomer.jar" />
    <resource-root path="./t24lib/AA_ARAccountsData.jar" />
    <resource-root path="./t24lib/AA_ARC.jar" />
  </resources>
</module>
```

- Add jar to the java Build path in DS



3. Create the class

```
WalkinCustomerOverride.java
package com.newbank;
import java.util.List;
import com.temenos.api.TBoolean;
import com.temenos.api.TString;
import com.temenos.api.TStructure;
import com.temenos.api.TValidationResponse;
import com.temenos.t24.api.hook.system.RecordLifecycle;
import com.temenos.t24.api.system.DataAccess;
import com.temenos.t24.api.tables.walkincustomer.WalkinCustomerRecord;

public TValidationResponse validateRecord(String application, String recordId, TStructure record,
    TStructure lastLiveRecord) {
    // validate phone and raise override
    DataAccess da = new DataAccess(this);
    String cusIds = "";
    List<String> recIds;
    int len;
    WalkinCustomerRecord WCR = new WalkinCustomerRecord(record);
    String phone = WCR.getPhone().toString();
    if (!phone.isEmpty()) {
        recIds = da.selectRecords("BNK", "CUSTOMER", "", "WITH PHONE.1 EQ '" + WCR.getPhone().toString() + "'");
        len = recIds.size();
        if (len > 0) {
            for (int i = 0; i < len; i++)
                cusIds = cusIds + " " + recIds.get(i);
            WCR.getPhone().setOverride("This phone number is used by CUSTOMER " + cusIds);
        }
    }
    // VALIDATE email and raise override
    String email = WCR.getEmail().toString();
    if (!email.isEmpty()) {
        recIds = da.selectRecords("BNK", "CUSTOMER", "", "WITH EMAIL.1 EQ '" + email + "'");
        len = recIds.size();
        cusIds = "";

        if (len > 0) {
            for (int i = 0; i < len; i++)
                cusIds = cusIds + " " + recIds.get(i);
            WCR.getEmail().setOverride("This email is used by CUSTOMER " + cusIds);
        }
    }
    return WCR.getValidationResponse();
}
```

4. Create the EB.API, VERSION

EB.API	WALKIN.CUSTOMER.OVERRIDE
Description	EN WALKIN CUST
Protection Level	Full
Source Type	Method
Java Method	validateRecord
Java Class	WalkinCustomerOverride
Java Package	com.newbank

Version	ST.WALKIN.CUSTOMER,DEMO
Records Per Page	1
Fields Per Line	Multi
Language Code.1	1
Field No.1	NAME
Text Char Max.1	35
Pre Syndication ID.1 .1	Name
Enrichm Char.1	35
Table Line.1	0
Field No.2	EMAIL
Text Char Max.2	35
Pre Syndication ID.2 .1	Email
Enrichm Char.2	35
Table Line.2	1
Field No.3	PHONE
Text Char Max.3	10
Pre Syndication ID.3 .1	Phone
Enrichm Char.3	10
Table Line.3	2
Input Routine.1	WALKIN.CUSTOMER.OVERRIDE

5. Launch the VERSION

CUSTOMER 100100 ↑ i - Ple

Phone No.1	206 553-6947
Mobile Phone Numbers.1	206 553-6947
Email.1	harrycrisp@gmail.com

St Walkin Customer Demo 829

Name	HARRY THOMAS CRISP
Email	harrycrisp@gmail.com
Phone	206 553-6947
Override.1	This phone number is used by CUSTOMER 100100
Override.2	This email is used by CUSTOMER 100100

= This phone number is used by
CUSTOMER 100100
This email is used by CUSTOMER 100100

CANCEL ACCEPT

Using EB.TABLE.PROCEDURES

EB.API *	WALKI ▼				
	Key	Description	Protection Level	Source Type	Java Method
	WALKIN.CUSTOMER.DEFAULT	WALKIN CUST	FULL	METHOD	defaultFieldValues
	WALKIN.CUSTOMER.OVERRIDE	WALKIN CUST	FULL	METHOD	validateRecord

EB.TABLE.PROCEDURES	ST.WALKIN.CUSTOMER
Description *	EN WALKIN CUSTOMER
Initialise Proc.1	
Check Id Proc.1	
Check Rec Proc.1	WALKIN.CUSTOMER.DEFAULT WALKIN CUST ▼
Prs Fld Proc.1	
Check Fld Proc.1	
Crossval Proc.1	WALKIN.CUSTOMER.OVERRIDE WALKIN CUST ▼

PGM.FILE	WALKIN.CUSTOMER.DEFAULT
Type	S
Product	EB

PGM.FILE	WALKIN.CUSTOMER.OVERRIDES
Type	S
Product	EB

Using EB.TABLE.PROCEDURES

```
public class WalkInCustomerOverride extends RecordLifecycle {  
  
    public String checkId(String arg0) {  
  
    @Override  
    public void defaultFieldValues(String application, String recordId, TStruct  
        WalkinCustomerRecord WCR = new WalkinCustomerRecord(record);  
        if (WCR.getEmail().toString().isEmpty()) {  
            WCR.setEmail("abc@abc.com");  
            record.set(WCR.toStructure());  
        }  
  
    }  
  
}
```

St Walkin Customer Demo 456

Name

Email

abc@abc.com

Phone

| Local Table

Lesson - 6

| DEMO Creating Local Table

- Create a local field called CUS.HOBBIES
- Attach field to CUSTOMER
- The local field gets its values from a LOCAL APPLICATION
- Use version to input hobbies for authorized CUSTOMER
- Minimum of 3 hobbies must be input

| 1. Creating the Local Application

- Create local application CUS.HOBBIES and input few records

EB.TABLE.DEFINITION CUS.HOBBIES	
Description	EN CUSTOMER HOBBIES
Product	ST
Field Name.1	DESCRIPTION
Description.1	HOBBY
Max Char.1	30
Data Type.1	ALPHANUMERIC
Prefix	ST.CUS86
File Type	Cus

List of Hobbies	
Deal Number	Description
DANCING	Dancing
GARDENING	Gardening
MOVIES	Watching Movies
ORIGAMI	Origami
READING	Reading Books

2. Create LOCAL.TABLE called CUS.HOBBIES

The screenshot shows the 'LocalFieldsDefinition.domain' editor. The 'Source' tab is active, displaying a list of fields. The field '[0-1] CUS.HOBBIES (EB_Foundation:ST_CUS_HOBBIES)' is highlighted with a yellow box. Below the list, the 'Association' section is visible, containing the following fields and options:

- Name:** CUS.HOBBIES
- Type:** EB_Foundation:ST_CUS_HOBBIES (highlighted with a yellow box)
- Modifiers:**
 - ☐ Primary Key
 - ☐ Business Key
 - ☐ Required
- Containment:**
 - ☐ By Value
 - ☒ By Reference (highlighted with a yellow box)
- Multiplicity:**
 - ☒ One
 - ☐ Many

A 'Browse...' button is located to the right of the 'Type' field.

3. Add the LOCAL.TABLE to CUSTOMER

The screenshot shows the 'X_CUSTOMER.domain' editor in a 'Source' view. The tree structure on the left lists the following elements:

- X_CUSTOMER (http://www.odcgroup.com/X_CUSTOMER)
 - Group_CUS_HOBBIES
 - [0-1] CUS_HOBBIES (EB_Foundation:ST_CUS_HOBBIES)
 - X_CUSTOMER (ST_Customer:CUSTOMER)
 - [0-1] CREDITOR_ID (T24BusinessTypes:A)
 - [0-1] CU_EFF_DATE (T24BusinessTypes:D)
 - [0-1] CUSTOMER_SEGMENT (EB_ModelBank:EB_CUSTOMER_SEGMENT)
 - [0-1] ESBASE_BIRTH_PROVINCE (T24BusinessTypes:AAA)
 - [0-1] FSCS_ACCT_STATUS (UKFSCS_Reporting:FSCS_ACCOUNT_STATUS)
 - [0-1] FSCS_ELIGIBLE (UKFSCS_Reporting:FSCS_ELIGIBILITY)
 - [0-n] GroupSample_CUS_HOBBIES (X_CUSTOMER:Group_CUS_HOBBIES) **(highlighted)**
 - [0-1] KYC_SOURCE (LocalFieldsEnumeration:LocalField_KYC_SOURCE)
 - [0-1] KYC_VERIFIER (T24BusinessTypes:A)
 - [0-1] NPFI_FLAG (T24BusinessTypes:A)
 - [0-1] TFN_EXEM_CATEG (LocalFieldsEnumeration:LocalField_TFN_EXEM_CATEG)

Below the tree, the 'Association' properties are displayed:

Association
Edits the core properties of the association.

Name: CUS_HOBBIES

Type: EB_Foundation:ST_CUS_HOBBIES Browse...

Modifiers	Containment	Multiplicity
☐ Primary Key	☐ By Value	☒ One
☐ Business Key	☒ By Reference	☐ Many
☐ Required		

4. Create the class

```
package com.newbank;

import java.util.List;
import com.temenos.api.LocalRefClass;
import com.temenos.api.TField;
import com.temenos.api.TStructure;
import com.temenos.api.TValidationResponse;
import com.temenos.t24.api.hook.system.RecordLifecycle;
import com.temenos.t24.api.records.customer.CustomerRecord;
import com.temenos.t24.api.system.DataAccess;

/** VERSION CUSTOMER,HOBBY is used to add a minimum of 3 hobbies for authorised customers */
public class CustomerHobbies extends RecordLifecycle {

    @Override
    public TValidationResponse validateRecord(String application, String recordId, TStructure record,
        TStructure lastLiveRecord) {
        CustomerRecord cusRec = new CustomerRecord(record);
        LocalRefClass lrf = cusRec.getLocalRef("CUS.HOBBIES");
        List<TField> hobbies = lrf.get();
        TField dummy = new TField("");
        if (hobbies.size() <= 2) {
            lrf.set(dummy, 0); // to overcome array index out of bounds when no hobby is entered
            lrf.get(0).setError("Please select minimum of 3 hobbies");
        }
        return cusRec.getValidationResponse();
    }

    @Override
    public String checkId(String idNew) {
        DataAccess da = new DataAccess(this);
        CustomerRecord cusRec = null;
        try {
            cusRec = new CustomerRecord(da.getRecord("CUSTOMER", idNew));
        } catch (Exception e) {

            throw new Error("This VERSION can be used to edit Authorised customers only or Invalid Customer ID");
        }
        return idNew;
    }
}
```

Create class , Export JAR and restart jBoss

| 5. Create the EB.API records

EB.API *

CUS.H ▼

Key	Description	Protection Level	Source Type	Java Method
CUS.HOBBIES	CUS HOBBIES	FULL	METHOD	validateRecord
CUS.HOBBIES.ID	CUS HOBBIES ID	FULL	METHOD	checkId

6. Create/Modify version

- Create the version CUSTOMER,HOBBY in DS and generate code. Modify the VERSION and add the hooks

Version CUSTOMER,HOBBY		
Records Per Page	1	
Fields Per Line	Multi	
Language Code.1	1	
Field No.1	MNEMONIC	
Text Char Max.1	10	
Enrichm Char.1	10	
Table Line.1	0	
Field No.2	CUS.HOBBIES-1	
Text Char Max.2	50	
Enrichm Char.2	50	
Table Line.2	1	
No Of Auth	1	
Multi Possible	Y	
Local Ref Field	LOCAL.REF	
Input Routine.1	CUS.HOBBIES	CUS HOBBIES
Report Locks	Yes	
Id Rtn.1	CUS.HOBBIES.ID	CUS HOBBIES ID

Launching the VERSION

TEMENOS CUSTOMER,HOBBY | DELL ▶

Customer Hobby 100283

Mnemonic *	DELL
Cus Hobbies.1	GARDENING Gardening ▼ +
Cus Hobbies.2	DANCING Dancing ▼ +
Cus Hobbies.3	READING Reading Books ▼ +

TEMENOS CUSTOMER,HOBBY | 919919 ▶

Runtime Exception Error This VERSION can be used to edit Authorised customers only or Invalid Customer ID

TEMENOS CUSTOMER,HOBBY | DELL ▶

Customer Hobby 100283

✓ 🔄 || ⬆️ ⓘ - Please Select GO

Mnemonic *	DELL
Cus Hobbies.1	GARDENING Gardening ▼ +

Please select minimum of 3 hobbies

Please resolve the errors below to proceed:

✗ Cus Hobbies.1: Please select minimum of 3 hobbies

| Course Summary

- We have seen
 - Need for T24 Extensibility
 - T types
 - How to attach VERSION hooks
 - Attaching Enquiry hooks
 - Creating and executing a multi threaded SERVICE
 - Creating a local table using EB.TABLE.DEFINITION and customizing the application
 - How to debug the Java code



TEMENOS

Learning Community

thank.you

tlc.temenos.com